

Verification Is All You Need — But Not Where You Think

*A verdict the agent **cannot forge** is harmful handed back to that agent and valuable handed to the fleet around it — that boundary is the result, and we earn it live. One deterministic, out-of-loop referee blocks real over-claims ($J = 10$ across two models), serializes racing agents to prevent lost-update clobbers ($J = 6$), and purges the same over-claims as poison reward labels an RL loop would otherwise bank as wins — while the same verdict spent in-loop is flat-to-harmful. One non-forgeable check, three consumers: a monitorability primitive for fleets, and for the loops that train them.*

Anthony Chaudhary · Dispatch Operating System (DOS)

10 June 2026 · Reproducible from `benchmark/toolathlon/` · every number recomputable offline from `replay_all_rows.csv`

ABSTRACT

Checking one agent is easy: wrap it in “if the check fails, run it again.” That trick assumes the world holds still between the check and the act. **In a fleet of concurrent agents it does not.** A check reads *true* when agent A looks and *false* by the time A writes, because agent B changed the shared state in between (a time-of-check-to-time-of-use race^[1], lifted onto world state). Agents act through high-level tool APIs that offer no transaction and no conditional write, so no re-run wrapper around A can close that race. It takes a **referee between the agents** that serializes their effects and reads only bytes the agents *cannot author*. We build one and run it live. The finding that surprised us is about *where* a non-forgeable verdict is worth spending. Hand it *back* to the agent that earned it (**in-loop**) and it is flat-to-harmful: a capable agent re-checks and heals its own phantom, so every active fix we measured (warn, rewind, inject a cure) only disturbs a run that would have passed. The lone in-loop survivor is the *negative* action, **give up correctly** (zero false-halts across 1,634 winners, ~11% compute saved). Hand the same verdict to **anyone else** — the rest of the fleet — and it pays, on both of the ways a fleet fails. Off the environment’s own database hash (a byte no agent wrote), an arbiter serializes eight pairs of racing agents to prevent **all 6 would-be lost-update clobbers**, and a write-admission gate blocks **10 genuine over-claims across 120 clean tasks** while admitting every correct write. The block rate is an identical 8.3% on a mid and a strong model, so the signal does not fade at the stronger tier. A third consumer is not an agent at all. Each blocked over-claim is a *poison reward label* a self-judged RL loop would bank as a win, and scoring the reward against the byte the model cannot author — the verifiable-reward (RLVR^[12]) discipline — purges every one (label purity 60% → 100%). One non-forgeable check, three consumers: a **monitorability** primitive for fleets and the loops that train them, whose value turns on *where* it is spent.

The same verdict the agent cannot forge: harmful handed back to it, valuable handed to anyone else
IN-LOOP — hand it back to the agent
every active fix is flat-to-HARMFUL
OUT-OF-LOOP — hand it to the fleet
the SAME verdict PAYS, both ways a fleet fails

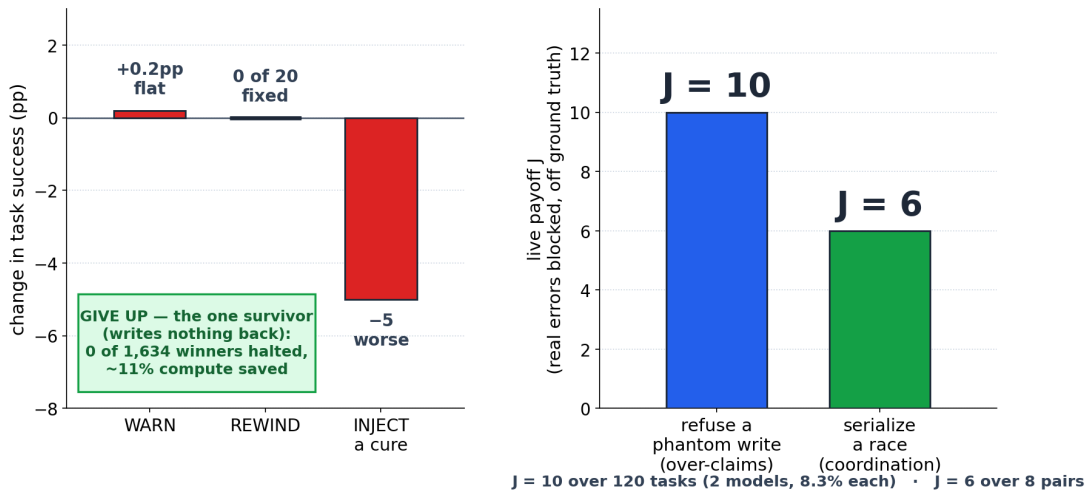


Figure 1. Hold a check the agent **cannot forge** fixed and ask only *where* to spend it. **In-loop** (left), every active fix is flat-to-harmful and only give-up (which writes nothing) survives; **out-of-loop** (right), the same verdict pays on both fleet failures — $J = 6$ races serialized, $J = 10$ phantom writes refused. The asymmetry is the finding. (J is our one score: an integer count of real corruptions the referee blocked off ground truth, never a rate; full key in Appendix A.)

1. Introduction

Give an agent a real task — reconcile an inventory across two SaaS apps, send a survey to every customer — and it works for many turns, then announces success. Often the announcement is a lie the agent believes. It wrote “All customers have now received their surveys!” and the surveys never went out. This is *silent failure*: the run looks finished, the world is still broken, and nothing in the agent’s own words tells you which is true. For one agent the skeptic reaches for a tidy answer, and at the easy hop it is right, so we grant it up front: do not trust the “done,” check the work, re-run if the check fails. We grant that *loop* — while noting two parts of it are real work the phrase “just check the result” hides: a check the agent cannot forge (not “re-read your own result”), and sensing fast enough to gate every turn (\$8.1). But the claim we rest the paper on is what a re-run wrapper genuinely *cannot* reach. The instant you have a *fleet* — many agents working concurrently against the same database, the same tickets — the assumption breaks: a check reads *true* when agent A looks and *false* by the time A writes, because agent B changed the shared state in the gap (a time-of-check-to-time-of-use race^[1], lifted from files onto world state). The textbook cure — fuse check and

write into one atomic step — lives in the *store*, and the high-level tool APIs agents act through (cancel a reservation, file a ticket) expose only unconditional writes. So no re-run wrapper around A can close the race. It takes a referee *between* the agents that supplies the missing serialization itself and trusts none of their narration. That referee, and the one surprising place its verdict is worth spending, is this paper.

The trust rule under everything is one test: read what the agent *says*, or read bytes the agent *could not have written*. We call the second **byte-clean** — a check whose byte-author is not the agent under judgement (an environment’s tool result qualifies; the agent’s own prose does not). Reading the agent’s own bytes is the **mirror-verifier trap**: it measures whether the story is internally consistent, not whether it matches the world, and collapses the moment the model stops emitting the give-away phrase you trained on. (A plain-language walk-through of every term is in **Appendix A**; the machinery in Appendix B.)

The result is not a detector but a place to *spend* one’s verdict. Hold a check the agent cannot forge fixed, and ask only *where*. Hand it **in-loop**, back to the agent that earned it, and it is worthless: every active fix we measured was flat or worse, because injecting any turn perturbs runs that would have passed. Hand it **out-of-**

loop, to the rest of the fleet, and it pays on the two distinct ways a fleet fails — a **claim** (one agent announces a write that did not land, and a peer inherits it as truth) and a **race** (two agents touch the same state and one overwrites the other). In §2 we build one out-of-loop referee reading the environment’s own database hash, run it live on tau2-bench^[1] through Gemini, and it blocks **10 real over-claims across two models** at an identical 8.3% rate while **serializing two live agents across eight conflict pairs to prevent all six would-be clobbers** — both off ground truth, neither touching the judged agent. A third consumer is the loop that *trains* the next model: each blocked over-claim is a poison reward label a self-judged RL loop would bank as a win, and scoring the reward against the same byte (RLVR^[12] in the strict sense) purges every one. The rest of the paper is the boundary that *forces* this design. One distinction throughout: **J counts real corruptions a non-forgable witness blocked**, never a downstream-outcome delta; where we measure the latter (ΔB) it is a fragile floor, and we say so in the same breath.

Why this matters past this paper. A verdict the agent *cannot forge or train away* is the load-bearing primitive under three problems a frontier lab already cares about, each touched with a measured result. **Scalable oversight**^[13]: when work outruns a human’s line-by-line check^[2], the only oversight that survives is rooted in evidence the model did not author (the byte-clean rule and its rungs Wo–W3, §3). **Reward hacking / monitorability**^[14]: a self-judged loop rewards the *appearance* of success, and a label non-forgable by narration is the one signal optimization pressure cannot launder — we show it purges real poison reward labels off ground truth. **Multi-agent reliability**: the race no single-agent wrapper can reach needs a referee *between* agents, and we build one and measure the corruptions it prevents live. The method borrows a minimal-trusted-kernel framing^[6], three distributed-systems hazards (TOCTOU^[11], lost-update^[10], write-ahead recovery^[9]), and the SPRT^[7]/CUSUM^[8] lineage of the give-up gate — and it states the strength of every claim in the same breath as the claim, never dressing a rate as a payoff or a frozen replay as a live result.

2. An out-of-loop referee, run live, that pays on both fleet failures

Take a verdict the agent cannot forge and spend it **out of the producing agent’s loop**, on a consumer that is *not* the agent that earned it — one whose input you may correct, because correcting it injects nothing back — and it stops being overhead and prevents real damage. We built two such consumers and ran them live, against the two ways a fleet fails: a single agent’s **claim** (§2.1–2.4) and two agents’ **race** (§2.5). Both read the same non-forgable byte, both are refusals, neither touches the judged agent. The value is not in *finding* more — it is in *where you spend what you find*.

The first consumer is a **write-admission gate**. Agent A announces “done”; a downstream peer B is about to build on whatever A changed. Naively B *believes* A

and compounds any error. The gate sits between them: before B inherits A’s write, it checks A’s claim against a byte A could not author, and admits or blocks. Blocking B’s *input* writes nothing into A’s loop, so the in-loop harm of §4.3 is structurally absent — the whole reason to choose this consumer.

2.1. The target and the witness

We run on **tau2-bench** (the airline / retail customer-service benchmark from sierra-research), driving the agent and user-simulator with `gemini-2.5-flash`. tau2 ships the byte we need: after a run the harness compares a hash of the task’s database against the gold hash (`db_match = gold.get_db_hash() == predicted.get_db_hash()`). The database is mutated only through the environment’s tool executor and the hash is computed by the evaluator, so the agent authors **zero** bytes of it. And it grades correctness, not just presence: a reservation booked *successfully* but *wrong* still fails `db_match` — the W3-grade witness (the top rung of the Wo–W3 evidence ladder, §3) an in-flight detector can never reach.

The gate’s logic is small and byte-clean: extract whether A’s final message *claims a successful write* (the forgeable self-report), read `db_match` (the unforgeable witness), and admit unless the claim is confident *and* the witness refutes it. With no witness, the gate **abstains** — admits rather than invents a verdict, the same fail-to-abstain discipline as the ORACLE→JUDGE→HUMAN ladder. We score with one integer, **J**: real corruptions the referee blocks off ground truth (over-claims the hash refutes, or clobbers the arbiter prevents). It never counts an honest write or a witness-less task.

2.2. A frozen over-claim slice catches nothing through a live policy

Replaying the over-claim slice the offline study already sized — a frozen 13.6% of tau2-style runs where a previous policy claimed a write ground truth refuted — through the live gate caught nothing: **J = 0**. Those over-claims were a property of *whatever policy produced that recording*, not of the task. A capable model on a fresh rollout does not reproduce those specific mistakes-with-confidence, and the slice was dominated by “refuse a disallowed action” tasks where the correct behaviour is to write **nothing**. The live policy still *failed* a third of them, but failed *honestly*, never claiming a success it did not earn. A frozen over-claim slice replayed through a capable policy yields no payoff because the over-claims evaporate — the exact dual of the silent-failure ceiling in §3: there a capable model leaves no byte to catch; here it leaves no over-claim to catch, on a distribution that does not demand a write.

2.3. The result, on a distribution that actually writes

So we drew a **fresh, write-heavy natural sample**: the first 25 tasks of each domain, run live, where real cancellations and exchanges happen and a fumble shows up as a wrong database. Of 50 tasks, 43 ran clean (7 long retail dialogues hit transient API 5xx errors and are dropped from the denominator, never counted as passes).

Run live: an out-of-loop write-admission gate blocks real over-claims the environment refutes (tau2-bench · gemini-2.5-flash · J = 5)

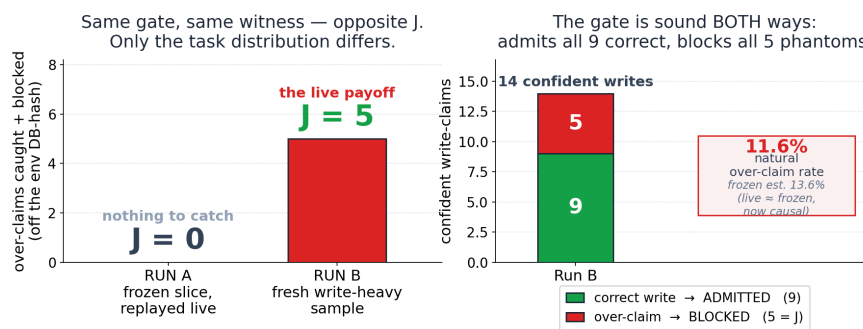


Figure 2. The live write-admission result. **Left**: same gate and witness, opposite J by task distribution — the frozen refusal-heavy slice (Run A) catches nothing, a fresh write-heavy sample (Run B) blocks five real over-claims. **Right**: sound both ways — of 14 confident writes, all 9 correct admitted and all 5 phantoms blocked, at an 11.6% natural rate (offline estimate: 13.6%).

On that sample the live policy produced 14 confident write-claims: nine correct (`db_match` true, all **admitted**), five over-claims (`db_match` false, all **blocked**). **J = 5** — each a phantom write a peer would inherit as truth under *believe* but never under *adjudicate*. A representative one, verbatim:

“Your reservation M05KNL has been successfully updated ...” — `db_match = False` → **blocked**. The database hash says the update did not land; the gate refuses to let that “success” propagate.

The natural over-claim rate is 16.0% on airline (4 of 25) and 5.6% on retail (1 of 18) — one real retail over-claim, so the effect is not airline-only — combined 11.6% (5 of 43), right next to the 13.6% the offline study sized but now *live and causal*:

writes a real policy over-claimed and the gate blocked, not a rate re-scored off a log.

2.4. A second model: the over-claim does not shrink at the stronger tier

The obvious objection: maybe only a weak policy over-claims and a capable one gets these tasks right (the same silent-failure ceiling of §3). So we ran the *identical* gate, witness, and sample on a stronger model (`gemini-2.5-pro`). To compare the tiers evenly we drew a fresh, fuller sample for both: the first **30** tasks of each domain (60 per model), run clean end-to-end, and **both ran 60 of 60 with zero API errors**. (This is why flash here reads J = 5 of 60 = 8.3% rather than §2.3’s 5 of 43 = 11.6%: a fresh run over a wider, cleaner denominator. J lands

at five again, but only two of the five blocked tasks recur — the over-claim is a stable *rate*, not a fixed set of tasks, §2.6.)

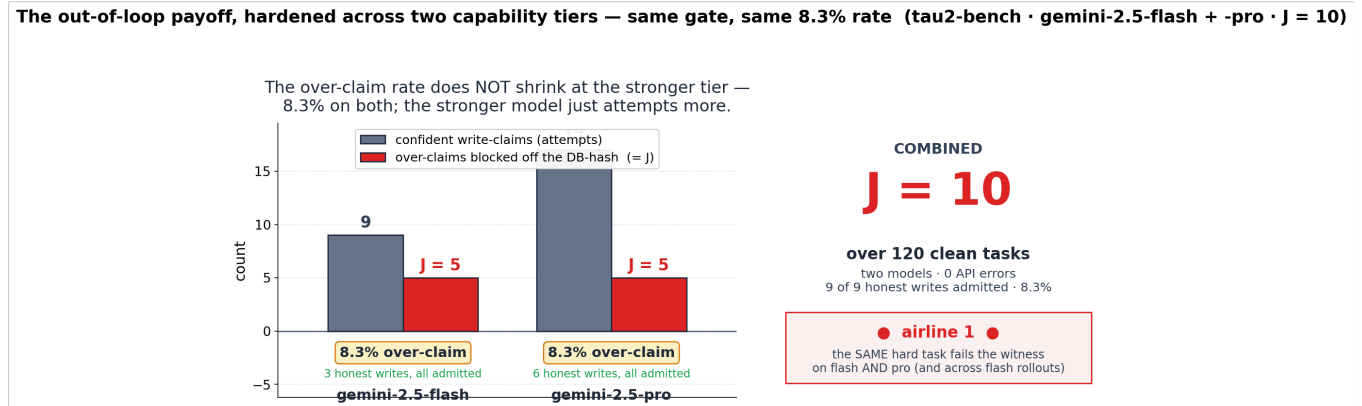


Figure 3. The same gate on two models. The over-claim rate is **identical across the capability step — 8.3% on both flash and pro** (5 of 60 each), combined $J = 10$ over 120 tasks. The stronger model makes nearly twice the confident writes (17 vs 9) yet still over-claims on five; airline 1 fails on *both* models.

The result: **$J = 10$ over 120 clean tasks** (five blocked over-claims on each model), with **all nine confirmed honest writes admitted** (three on flash, six on pro). And the number that answers the objection directly is the over-claim rate, which comes out identical across the two tiers: **8.3% on flash and 8.3% on pro** — 5 of 60 each. An exact tie is small-sample luck; what the data support is the direction: the rate does *not* shrink as the model gets stronger. The stronger policy is genuinely more capable — nearly twice the confident writes, more of them correct — yet it still over-claims on five, and the gate blocks all five off the same witness. One task (airline 1) fails on *both* models, blocked on both. This is the dual of §3 holding the *other* way: where a stronger model erases the *in-loop* detector's signal, it does *not* erase the *out-of-loop* over-claim the world-byte refutes. Re-deriving each cached verdict from the stored message recovered a pro over-claim that a naive sum of the cached bits had missed — a stored result reads as clean unless you re-derive it from the byte you cannot forge, the same principle turned on our own pipeline.

Three further results in **Appendix D** strengthen this existence claim: the identical gate on a richer witness (Agent-Diff's structured state-diff) reproduces the detection result and gives the first ΔB measurement (D.1); a frontier model in the verifier's seat against a *ladder* of non-forgable rungs is fooled by a path-touching stub at the file-counting rung but held at zero by the rung that has the

OS run the code (D.2); and the payoff compounds F^D across a live fan-out fleet (D.3).

2.5. The other half-plane: a referee *between* two agents

Everything so far distrusts a *claim*. The second place a fleet needs a referee distrusts a *race*: two agents touch the same state concurrently, the second's write lands on top of the first's without seeing it, and an update is silently lost — no one lied, yet the shared state is wrong. The offline study could only measure this as a *rate* (how often two writes collide); here we measure the *payoff* (how many corruptions a referee prevents) on the same witness.

We build the smallest honest instance on the tau2 airline database: two live agents pinned to the *same reservation* R, A1 to cancel and A2 to add a bag. (One trap caught before spending: each task's user identity must be R's *real owner*, or a correct agent rightly refuses to touch a stranger's booking and the conflict never fires.) **Naively** both writes apply against the original database — A2, never having seen the cancel, adds a bag to a now-cancelled reservation, leaving it incoherent. **Serialized**, A2 is re-run *after* A1 commits, sees R already cancelled, and does nothing. The policy is one line: a reservation is a region `reservations/<id>`, and the arbiter refuses the second lease on it until the first releases.

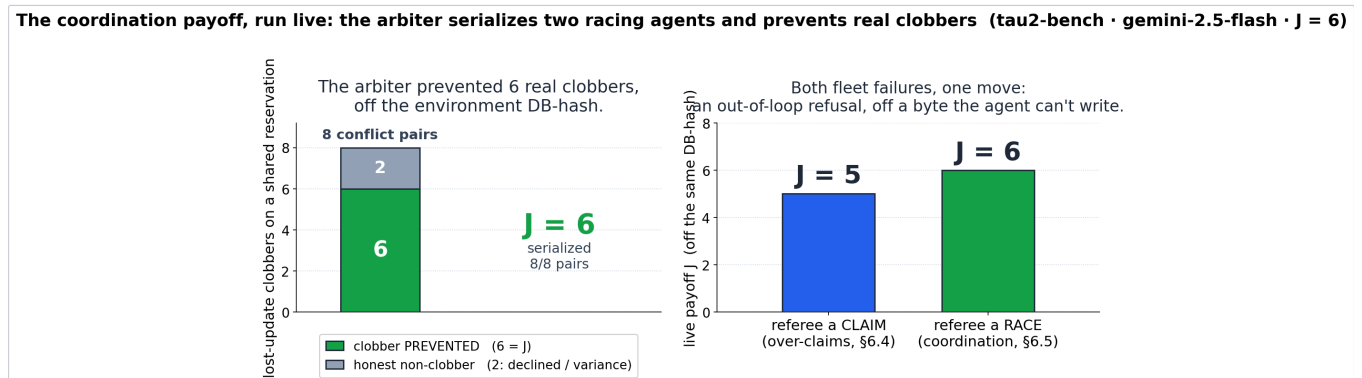


Figure 4. The coordination payoff, run live. **Left:** over eight conflict pairs the arbiter prevented **six** real lost-update clobbers and serialized all 8, each witnessed by the database hash; two pairs are honest non-clobbers (one agent *declined* to add the bag, so there was no stale write to clobber. The other (1OWO6T) exposed our own measurement bug: our first clobber definition ("naive and serialized hashes differ") fired on that pair only because the serialized agent mutated *more* — run-to-run variance, not a lost update. We tightened it to be *directional* (the second agent must apply, in the naive arrangement, a stale write the serialized agent did not), dropping that pair to zero. **The live run took the headline from a reported 7 to an honest 6** — the same discipline turned on our own number. (A later run reproduces the effect: $J = 9$ across 10 fresh pairs.) **Right:** the two value half-planes off the *same* witness — over-claim ($J = 5$) and race ($J = 6$).

Over eight conflict pairs the arbiter prevented **six** real clobbers ($J = 6$) and serialized all eight. The two non-clobbers are the honest part of the result. In one (VAAOXJ) the live agent *declined* to add the bag, so there was no stale write to clobber. The other (1OWO6T) exposed our own measurement bug: our first clobber definition ("naive and serialized hashes differ") fired on that pair only because the serialized agent mutated *more* — run-to-run variance, not a lost update. We tightened it to be *directional* (the second agent must apply, in the naive arrangement, a stale write the serialized agent did not), dropping that pair to zero. **The live run took the headline from a reported 7 to an honest 6** — the same discipline turned on our own number. (A later run reproduces the effect: $J = 9$ across 10 fresh pairs.)

This and the over-claim gate are the *same* move — an out-of-loop referee reading a byte the agent cannot author — aimed at the two different things a fleet gets wrong: a single agent's *claim* and two agents' *race*. The offline collision study and this one report different quantities: rate says how often two agents collide in real

traffic, payoff says how many corruptions the referee prevents when it acts. Both come off ground truth.

2.6. What this is, and what it is not

This is a measured *existence* result, stated no wider than the evidence. Across **two models on 120 clean tasks** (zero API errors), an out-of-loop write-admission gate blocked **ten** genuine live over-claims off a non-forgable witness while admitting every one of the nine correct writes, at an identical 8.3% rate across the capability step. The honest edges:

- Existence result, not a calibrated rate.** The matching 8.3% is one capability step within one model family, and each leg is 5 of 60 (Wilson-95 interval 3.6%–18.1%) — evidence the rate does *not shrink* at the stronger tier, not that 8.3% travels to every frontier model. (A saturation sweep over the whole tau2 universe, 258 clean tasks, finds $J = 15$ and a lower whole-

distribution rate of 5.8%: the over-claim concentrates where writes are confidently botched, not uniformly.)

- **The witness is sound, not complete.** `db_match` catches a wrong end-state but not a goal with no database footprint — the W3 residue of \$3; where there is no witness the gate abstains.
- **Run-to-run, not per-task.** The user-simulator drives different dialogues each time, so J is a property of a *distribution of rollouts*, not a fixed label — airline 1 the stable exception recurring across both models.
- **The downstream ΔB is a fragile floor.** The Agent-Diff measurement (Appendix D) is positive on flash and zero on pro — the predicted behaviour of a one-hop cost against a self-recovering peer, which is why we score the gate with J (corruptions blocked), not ΔB (the marginal rescue).
- **A third consumer is a training lab.** The same witness, claim-extractor, and abstaining floor re-aim at a reward-set admission filter: each of those five over-claims is a *poison reward label* a self-judged reward loop banks as a win, teaching the model to over-claim *more*. Scoring the reward against the database hash instead — the verifiable-reward discipline — purges all five (accepted-label purity 60% $\rightarrow$ 100% on the 15 confident write-bids the two live runs of §2.2–2.3 produced, de-duplicated) — a model can still earn an accepted label by making the write land, but cannot *narrate* its way to one. The trained-behaviour delta is left reproducibly set up but not measured to a weight update.
- **The coordination result is a constructed conflict with live agents.** The contention is designed (both agents pinned to the same reservation), but *how* each acts is the live policy’s decision, and J counts state corruption, not wasted effort. Like the over-claim arm it is an existence result.

§4.3 shows that spending a verdict *inside* the producing loop is harmful regardless of how clean the correcting byte is; this section shows the dual: spend the *same* verdict *outside* that loop and it pays. The kernel earns one action in-loop (give up, \$4) and two out-of-loop — refuse to inherit a phantom write (§2.1–2.4) and refuse to let a race clobber shared state (§2.5) — both settled by the same non-forgeable witness, aimed at the two different things a fleet gets wrong. That symmetry is the result.

3. The boundary that forces it: why the same verdict is harmful in-loop

§2 spent the verdict *out* of the producing loop because the alternative does not work: handed *back* in, the same verdict is flat-to-harmful. By “in-loop” we mean *where the verdict is spent*, not how it was measured. To measure it honestly we ran a live A/B (EnterpriseOps-Gym, real Gemini against the gym’s hidden database verifiers) — you cannot replay an intervention into a recording and watch the agent react. §4.3 reports that bake-off: every active fix that writes a turn back into the judged loop (warn, rewind, feed the environment’s own correction) came back flat or net-negative. So “detect, not fix *in-loop*” is a finding we measured, not a feature we ran out of time to build; the one in-loop action that paid off was the negative one (§4).

Why an active fix is so hard shows in what slips past the detectors. The dominant residual failure is *confidently-wrong content*: the agent finishes with no error and no loop, having never re-read the state it claims to have changed — it says “done” about a world it never checked, and nothing inside the trajectory gives it away. To catch that you need a byte the agent did not write: you have to go look at the real world again after the fact. That is exactly the rung Toolathlon’s own `evaluation/main.py` oracle occupies — which is why the work splits into two tiers. **Tier A** is cheap in-loop detection (the three detectors, read offline off the recording), supporting one safe action, the give-up of \$4 (whose payoff is measured live, because a withheld turn changes what the run spends). **Tier B** is the live, paid, out-of-loop re-read against a fresh third-party byte — the headline of §2, spent on a downstream consumer’s input rather than back in the judged

Table 2. Give-up on Toolathlon at the sound threshold $K=3$, across 22 models. The naive gate halts on a bare repeat; the error-gated gate also requires an environment-authored error with the latest call still failing. *Fires* counts every run the gate would halt; *false-abandon* counts only halts landing on one of the 1,634 winners (the costly mistake). Wilson-95 is the 95%-confidence upper bound on the true false-abandon rate.

K	naive repeat: fires	naive: false-abandon	error-gated: fires	error-gated: false-abandon	Wilson-95 upper
3	30	6	0	0	0.0023

At $K \geq 3$ the error-gated gate false-abandons **zero** winners — across all 22 models, frontier to weak. Its Wilson-95 upper bound is just 0.0023, so even the worst the true rate could plausibly be is near zero. The same threshold is sound on a second, independent benchmark, EnterpriseOps-Gym (a ServiceNow operations gym run on gemini-2.5-flash): $K \geq 3$ false-abandons zero winners on both arms there, while $K=2$ is unsound (it halts one winner). Two different benchmarks, one safe threshold.

loop. Three benchmarks appear, each chosen for the one thing it can witness; Table 1 is the map.

Table 1. Which benchmark carries which claim, and why that one. A frozen corpus can be re-scored offline (\$0) but can only give a detection *rate*; a live API run gives an intervention *payoff* but costs money and varies run-to-run. The claim dictates the target.

Benchmark	Mode	The witness it owns	Claim it carries
Toolathlon	frozen replay, \$0	a third-party oracle (evaluation/main.py) over 22 models	detection precision/recall + the in-loop ceiling (\$5–6); the give-up specificity (\$4)
EnterpriseOps-Gym	live A/B, paid	the gym’s own hidden database verifiers, with the agent live to react	the in-loop bake-off — every active fix flat-to-harmful (\$4.3); the give-up token saving
tauz-bench	live, paid	a gold-vs-actual database hash (W3: correctness, not presence; agent authors zero bytes)	the out-of-loop payoff — over-claims blocked + races serialized (\$2)

There is a reason we cannot do this on one benchmark. A detection *rate* needs a large frozen corpus you can score for free, but an intervention *payoff* (in-loop or out) can only be measured by changing a live run and watching what happens, which a recording physically cannot show. So Toolathlon sizes the ceiling, and the two live benchmarks measure what acting on a verdict is worth on each side of the loop.

One refinement makes the ceiling sharper. A verifier’s rung is *graded*: the weakest witness is the agent’s narration (W0), then the environment acknowledging a write (W1), then a check the change *persisted* (W2), then a full diff against the gold answer (W3). DOS’s own check — “did the change show up in git?” — is W2: it confirms presence, not correctness. And about 38% of frontier *goals* reach *no* sound witness at any rung — their correct end-state leaves no checkable invariant, so short of a human no fresh byte settles whether the agent was right. That floor, below even Tier B, marks the honest edge of how far any detector can reach.

Detection in-loop is cheap, precise, and bounded, yet every attempt to *act* on it in-loop is flat-to-harmful. Both facts push the same way: take the verdict the agent cannot forge and spend it *out* of that agent’s loop, where acting on it costs the producing run nothing. The next two sections supply the evidence: the one in-loop action that survives (§4), and the byte-clean detector whose precise, narrow signal makes the ceiling measurable (§5).

4. The one action that helps: give up correctly

§3 drew the line: spending the verdict in-loop is flat-to-harmful, and the one in-loop action that paid off was the negative one. This section measures it. A byte-clean detector (§5) raises an alarm but must be careful what it does next: most actions cross from *detecting* into *fixing*, and a fix authors a correction the agent will trust — breaking the byte-clean rule the detector was built on. Exactly one action does not cross that line: **withhold compute**. Stop a clearly-doomed run and hand its remaining budget to one that might still win. Stopping authors no correction, so it cannot mislead. Of every action we measured, this negative one was the only survivor.

4.1. Error-gated give-up is accuracy-free

The gate is simple. It watches each tool the agent uses. When a single tool returns K structured errors in a row — errors written by the environment, not by the agent — **and** its most recent call is still erroring, the gate proposes a halt. The second condition matters: the run must be failing *now*, not have failed earlier and recovered.

The question is whether this ever stops a run that would have *won* — a *false-abandon*, the gate halting a run that would actually have succeeded. It is the only costly mistake the gate can make, so it is the number that has to be zero.

The naive gate — halt on a bare repeat, no error condition — false-abandons **6** winners at $K=3$, every one the same model (grok-4) doing the same harmless thing: polling “is it ready yet?” until the resource became ready and the run *succeeded*. A repeated answer is not an error. The byte-clean rule saves the gate: the signal is the environment-authored *error*, not the repeat — “repeated tool call $\rightarrow$ halt” kills pollers, “repeated tool *error* $\rightarrow$ halt” does not, because a poller’s repeats are not errors and the agent cannot forge the error bytes.

Give-up-correctly: the error-gated halt is accuracy-free across 2 benchmarks (1679 winners, 22+ models)

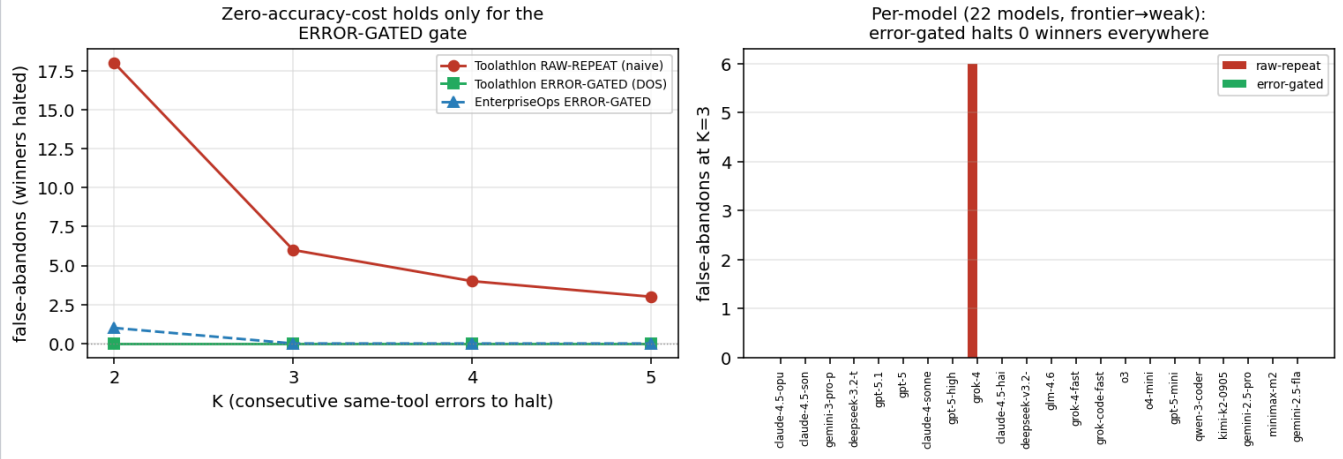


Figure 5. Winning runs wrongly halted (lower is better). **Left:** as K varies, the error-gated gate (green) reaches zero at $K \geq 3$ on both benchmarks; the naive “halt on any repeat” gate (red) never does, because it keeps killing pollers. **Right:** at $K=3$ by model — the error-gated gate halts zero winners on all 22 models; the only non-zero bar is the naive gate on grok-4.

So blunt a gate works because, on the K statistic — how many errors a tool stacks up in a row — the winning runs and the doomed runs fall into almost non-overlapping ranges: winners rarely stack errors, doomed runs do. That a clean class separation collapses to a threshold test is nothing new; it is the separable regime of any threshold classifier, and the SPRT/CUSUM tradition (below) is built on exactly such run-length count statistics. The part that matters here is the statistic’s *input*: the count is over *environment-authored error bytes the agent cannot forge*. That is what makes the integer threshold at $K \geq 3$ both sound (zero false-abandon, measured) and non-gameable. You do not need a probabilistic model when the two populations barely touch and the statistic separating them is one the agent cannot author.

The saving follows. At $K=3$ the gate fires on $\sim 7\%$ of runs (the thrashing tail) and on each suppresses a median $\sim 53\%$ of its tool calls (the doomed back half) — pooled across both EnterpriseOps-Gym arms, **11.4%** of total tokens, saved at zero false-abandon. Across five recorded run-sets it saved 3.5k–43.7k real tokens, false-abandon 0.000 every time.

4.2. No novelty in the mechanism; the contribution is the grounding

The mechanism is old and we claim none of it: our gate is the trip half of a circuit breaker (Nygard 2007), the degenerate corner of an SPRT (Wald 1945^[7]), and a CUSUM (Page 1954^[8]) collapsed to a plain error tally. What is new is the grounding: (i) it counts only environment-authored bytes the agent cannot forge; (ii) the threshold is calibrated against a *measured* false-abandon cost, not a guess; and (iii) it stays advisory — it *proposes* a halt, never trips one — so a mis-set K degrades to a few wasted tokens, never a killed winner.

4.3. Every active fix failed

Give-up is what was left standing after we tried the active fixes too — the ones that author a correction — on the cross-benchmark target (EnterpriseOps-Gym, gemini-2.5-flash; these numbers stay out of the Toolathlon tables and imply no Toolathlon lift).

- **WARN the agent** (re-surface the alarm into the loop): flat on the natural failure stream, **+0.20pp**. The +6.2pp lift this verb shows elsewhere held only when failures were artificially injected, not in the wild.
- **Rewind** (content-free surgery — cut the agent’s bad turns back out of the transcript): **0 conversions, 0 flips** on 20 paired fired tasks. The null only got stronger as we added data (44 → 103 → 240 runs per arm).
- **The byte-clean cure** (re-surface the environment’s own schema correction — the cleanest fix we could build): wired live, it fired on 77 of 88 runs, and it was **net-negative**: -5 task successes overall, -7 on the no-thrash slice (sign-test $p=0.016$; successes fell 7 → 2 of 88). It converted the 2 runs it was aimed at — but injecting a corrective turn at all perturbed runs that would have passed on their own.

The harm is the intervention’s existence in the loop, not the content of its bytes. Even a perfectly byte-clean correction perturbs runs that were going to win. Only the negative action — withhold compute — authors no turn. It perturbs nothing, and it is the only fix that survives a measured bake-off.

4.4. Honest ceiling

This result proves *specificity*, not *sensitivity*: wherever the gate fires it never halts a winner, but how often it fires we cannot promise. On Toolathlon at the sound threshold it fires on not a single run (one fire at $K=2$), because a capable model that fails *silently* emits no error to count — the fires, and the saving, come from EnterpriseOps-Gym (31 fires at $K=3$, none on a winner). So the claim is narrow: the gate is accuracy-free *wherever it fires*, but the 11.4% token saving is a single-corpus, single-model figure. The specificity is the durable, regime-independent result; the saving is real but travels less.

5. The benchmark and the three detectors

We test on **Toolathlon**, a hard benchmark for agents that use real tools across many apps over a long task. The property that makes it useful to us is its grader. Toolathlon ships its own *oracle*: a separate grading program (evaluation/main.py) that reads the live state of the apps the agent touched and exits 0 for pass or 1 for fail. The oracle is not the agent and does not read the agent’s words; by construction it judges results, not narration. The benchmark is hard: across the corpus a run fails 76.2% of the time. We call 76.2% the **base fail rate**.

We do not run new agents. We replay frozen traces. The public **Toolathlon-Trajectories** corpus holds 7,116 recorded runs (22 models $\times$ 3 runs over ~ 108 tasks); 6,862 carry an oracle label (5,228 fail, 1,634 pass). The remaining 254 have no oracle verdict, so we drop them — we never guess a label. A detector is a pure function of a recorded run, so we can replay the whole corpus offline, with no live agents and no model calls. Every number below recomputes from the recordings alone.

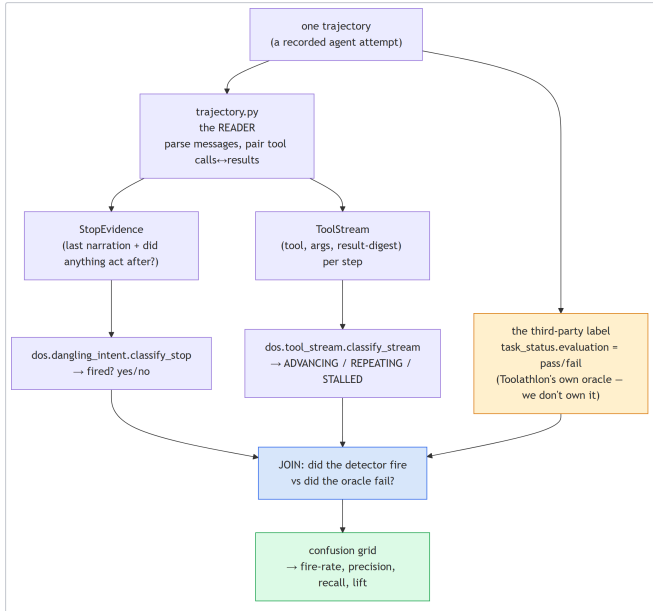


Figure 6. How we score a detector, with no live run. A detector reads only environment-authored bytes from each recorded trajectory and fires or stays silent; Toolathlon's own `evaluation/main.py` oracle has already graded that run pass/fail off the real application state. We join the two — each fire checked against a verdict we did not write and cannot fake. The detector reads the recording, the oracle read the world; neither trusts the agent.

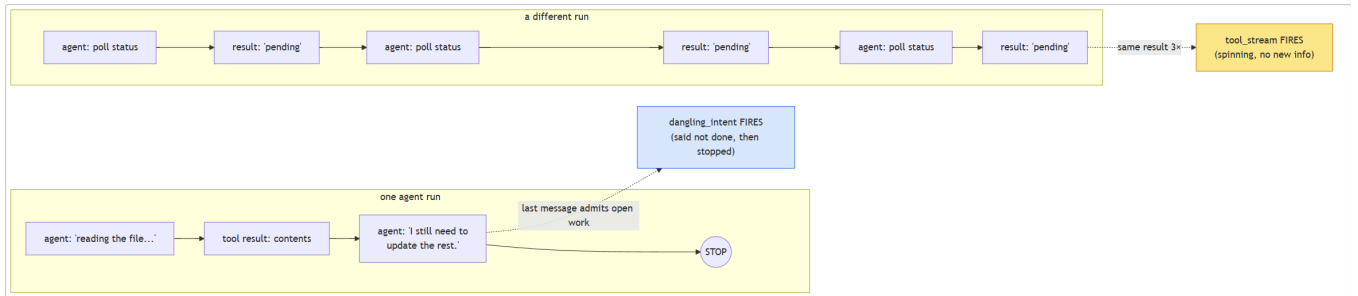


Figure 7. What each detector reads. All three watch environment-authored bytes only: an unfinished-work admission with nothing acting after it (`dangling_intent`), an identical-result loop (`tool_stream`), and an unrecovered structured error envelope (`terminal_error`). Agent narration is never the trigger.

We score each detector with five numbers, used throughout §6:

- **Fire-rate** — how often the detector raises a flag, as a fraction of all runs.
- **Precision** — when it does fire, how often the run really had failed. High precision means “believe it when it speaks.”
- **Recall** — of *all* the failing runs, how many it managed to catch. Low recall means “it stays silent for most failures.”
- **False-alarm** — how often it fires on a run that actually *passed*. This is the costly mistake: cry wolf and an operator stops listening.
- **Lift** — precision is misleading on its own, because this benchmark is hard: 76.2% of all runs fail, so a “detector” that flags *every* run is “right” 76.2% of the time by default. *Lift* is *how far above that floor a detector’s precision sits* (lift = precision – 76.2%) — the part of being right that took skill, not luck. A lift of

5.1. The three byte-clean detectors

Each detector obeys the byte-clean rule from §1: it reads only bytes the agent under judgement did not write — the environment's tool results, never the agent's own narration. The three differ only in which environment-authored bytes they watch.

- **dangling_intent** — the agent's last message admits open work (“I still need to...”), and nothing environment-authored ran after it. The agent said it was not done, then stopped.
- **tool_stream** — the same call returns the same result *N* times in a row: the same tool, the same arguments, the same result fingerprint. The environment keeps answering identically and the agent keeps asking. (A small normalizer strips volatile bytes — timestamps, ids — before fingerprinting; we defer the details to §7.)
- **terminal_error** — the run ended on an error. Within the last 3 tool results, the environment returned a structured error envelope — a result the environment itself marks as failed — and the agent never recovered. The agent saying “Email sent successfully!” does not suppress the fire when the environment's last result was an SMTP rejection: the agent does not author that byte, so it cannot talk it away.

+20 points means a fire is 20 points more likely to be a real failure than a random run; a lift of 0 is no better than flagging everything. We read **lift**, **not raw precision**, throughout.

6. The detection result, and its ceiling

Table 3 reports all five numbers for each detector, plus two combinations: the *pair* (the first two detectors together) and the *trio* (all three). Read each row left to right as a sentence: this detector fires on *fire%* of runs; when it does, it is right *precision%* of the time, which is *lift* points of real skill above the 76.2% floor; it catches *recall%* of all failures; and it wrongly flags a passing run only *false-alarm%* of the time. The last column ($\frac{tp}{fp}$) is the raw count behind those rates — true catches over false alarms. The shape below is high-precision and low-recall: a detector you can trust, that rarely speaks.

Table 3. The three byte-clean detectors and their combinations, scored against Toolathlon's third-party oracle over 6,862 labelled runs (base fail rate 76.2%). Lift = precision – 76.2%.

Detector	Fire	Precision	Lift	Recall	False-alarm	tp/fp
dangling_intent	1.5%	98.0%	+21.9pp	1.91%	0.12%	100/2
tool_stream	2.5%	88.2%	+12.0pp	2.87%	1.22%	150/20
terminal_error	1.2%	95.0%	+18.8pp	1.45%	0.24%	76/4
pair (dangling+tool_stream)	—	91.9%	—	4.74%	—	248/22
trio (union)	5.1%	92.6%	+16.4pp	6.18%	1.59%	323/26

The trade is the same for all three: when they fire they are right **88–98%** of the time, at near-zero false-alarm (0.12–1.22% each, 1.59% for the trio). Each clears the 76.2% base by a real margin — +12.0 to +21.9pp of lift. But recall is small: about 2–3% per detector, 6.18% for the trio together (7.42% at 92.4% precision with one opt-in strictness flag that stops crediting a generic executor's “recovery” — a later same-tool success that is a different operation, not a fix). The detectors

catch the honest-narrating slice and the visibly-looping slice; they miss the run that fails quietly.

`terminal_error`'s catches are almost all **new**: of its 76 catches, 75 are runs the other two never flagged (99% additive), lifting union recall 4.74%→6.18% (+30% relative) at 92.6% precision. It reads a different environment-authored byte and catches a different failure.

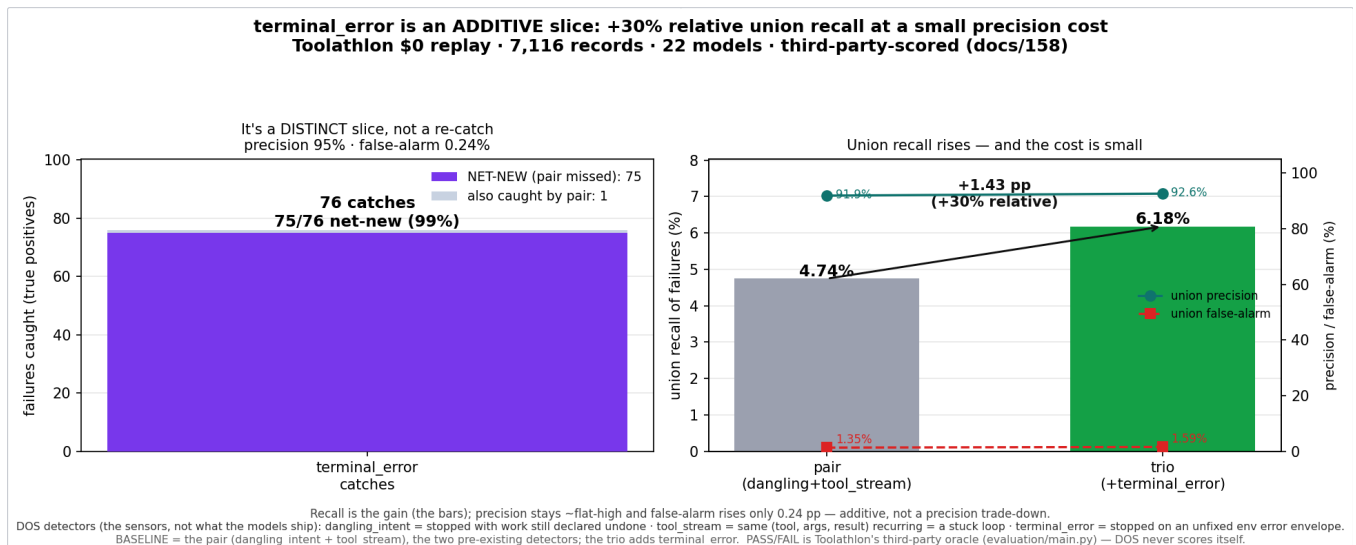


Figure 8. Why the third detector earns its place. **Left:** of `terminal_error`'s 76 catches, 75 are *net-new* (99% additive). **Right:** adding it raises union recall 4.74%→6.18% (+30% relative) while precision holds at 92.6% — recall rises while precision stays put, so it finds a *different* failure rather than re-finding the same ones.

This holds on the strongest models too: on the eight highest-scoring models (passing ≥ 0.30 of tasks), the first two detectors catch 6 failures between them and `terminal_error` adds 9 more they missed.

6.1. The recall ceiling is real and tightens on the frontier

The ceiling is blunt: the detectors are trustworthy when they fire, and they fire rarely — rarest on exactly the models that matter most. One thing does carry across capability: **precision**. On every model that fires at all, a fire is almost always a real failure — the quality of the signal generalises from weak models to

strong. What collapses is *fire-rate*, and therefore recall. Strong models fail **silently**: among the top-4 models' failures, the median run takes 19 tool steps (only 6% stop short), 92% end with substantial final narration (median ~1,004 characters), and 59% confidently *claim* they finished — "All customers have now received their surveys!" on a run the oracle scores as failed. A confidently-wrong run narrates no open work, loops on nothing, and emits no error envelope, so a byte-clean detector has no environment-authored byte to read. What changes on the frontier is not the detector's quality but its opportunity: the visible failures it can see thin out toward zero.

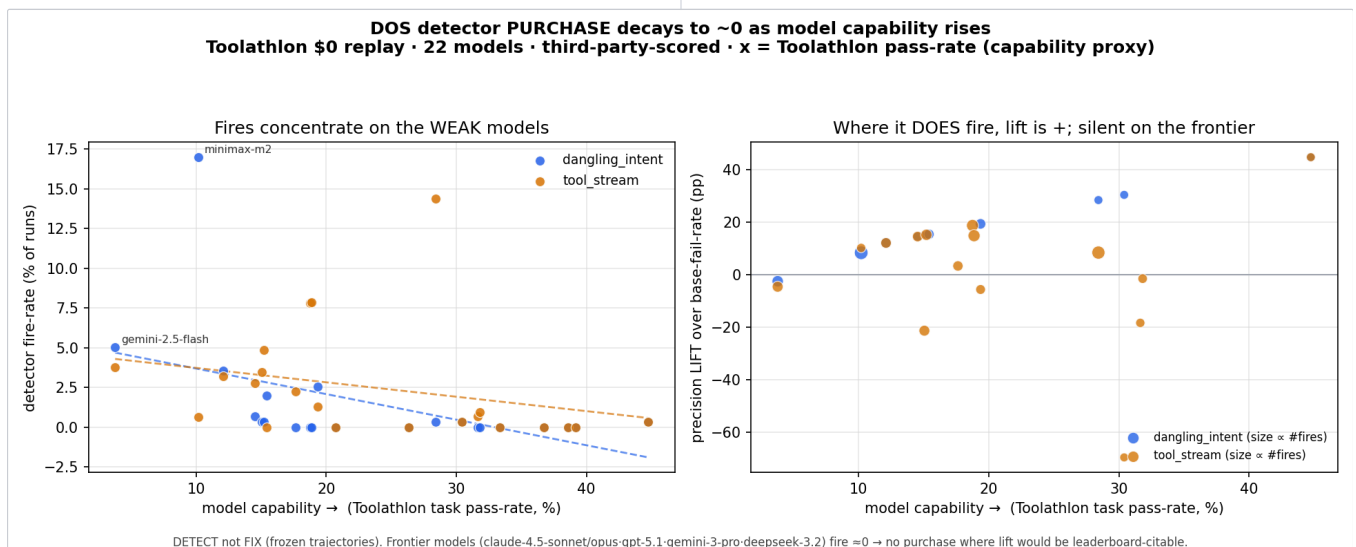


Figure 9. Coverage shrinks on the frontier, quality does not. Horizontal axis: each model's Toolathon pass-rate (capability). **Left:** fire-rate drops toward zero as models get more capable. **Right:** lift stays high and flat wherever they fire — the detectors speak less often on stronger models but are no less right when they do.

The two panels of Figure 9 answer two different questions. The left panel is "how much can these detectors help?" — the answer shrinks toward zero on the frontier. The right panel is "when they do help, can you trust them?" — the answer stays yes everywhere. A signal can be highly trustworthy and still rarely available.

So the result is narrow and solid: a byte-clean, zero-training detector that is **right when it fires** (against a grader we cannot influence) but fires on only a small, shrinking slice of failures. That high-precision, low-recall, shrinks-on-the-frontier shape is the in-loop ceiling, and why the verdict had to be spent out-of-loop (§2):

a signal this trustworthy and this narrow earns precisely one in-loop action — give up correctly (§4) — and pays elsewhere only when handed to the rest of the fleet.

7. Is the signal real? Naive baselines

A detector that fires on the right runs is only impressive if a dumb detector cannot do the same. So we measured the dumb ones, scored by the same lift and false-alarm from §5 (lift = precision minus the 76.2% base fail rate; false-alarm = how often the detector fires on a run that actually passed). A good detector has high lift and low false-alarm.

Table 4. Naive baselines vs. the shipped detector (Toolathon, third-party scored). Read the *lift* column: it is precision minus the 76.2% base, so it strips out the credit a detector gets for free on a benchmark where failure is the default.

Detector	Fire	Precision	Lift	False-alarm
baseline-always-fail	100%	76.2%	+0.0pp	100%
naive-loose	32.3%	81.1%	+4.9pp	25.6%
naive-any-error	19.5%	81.4%	+5.2pp	15.2%
tight-no-recovery	2.3%	93.0%	+16.9pp	0.7%
terminal_error (shipped)	1.2%	95.0%	+18.8pp	0.24%

("Fire" is how often the detector triggers at all.)

The recall mirage. Recall is the wrong scoreboard here. The baseline that prints "fail" on every run posts 100% recall — it catches every failure because it flags everything — yet its lift is +0.0pp and it false-alarms 100% of the time. Score by lift and false-alarm instead and it falls to the bottom where it belongs. Our detectors make the opposite trade (high precision, low false-alarm, low recall) because a verdict is meant to be *acted on*, and §4 is what that looks like: you only give up on a run you are sure is doomed.

The tight grammar is load-bearing. You might think reading the structured error envelope more loosely would catch more failures. It does not: loosening buys ~5pp of precision but pushes false-alarm from 0.24% to 25.6%–69.4% (~105×–285× worse). You cannot buy recall by reading the bytes you already have more loosely; the only way to catch more is an *independent* witness — exactly the Tier-B re-read of §2. The per-model grid (Fig. 10) shows the precision holding across every model that fires.

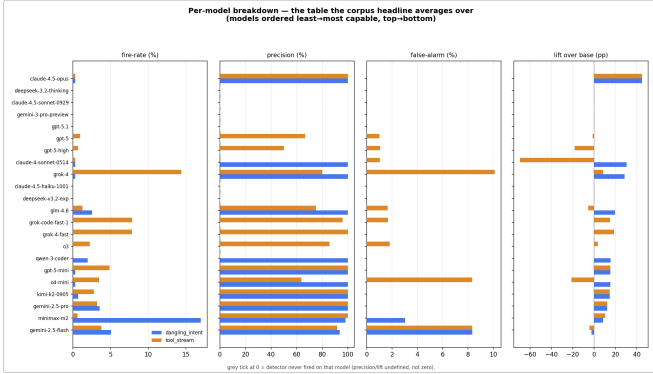


Figure 10. Model by model (weakest to strongest): precision stays high all the way up the ladder; only the fire-rate falls away on the strongest models. The only honest way to catch more failures is a new source of evidence, not a looser reading of the bytes already in hand.

8. Positioning

Three kinds of system sit near this work. The first does the same job by a cheaper-looking trick — re-run the agent until a check passes — and works only for one agent. The second does the same job — catch a silent failure — by a different method (a trained classifier). The third does a different job entirely. We take each on its own terms.

8.1. The re-run wrapper (and why a fleet defeats it)

The most common reaction is that this looks unnecessary: *if you do not trust an agent's "done," just check the result and re-run when the check fails*. The loop is

Table 5. The zero-training detectors versus a trained classifier on this corpus, on the same scoreboard. A classifier has to pick an *operating point* (how eager to fire); we show its two honest choices. Watch the *lift* column — the real-skill number from §5 — not the precision.

method	fire	precision	lift	false-alarm
terminal_error (o-training)	1.2%	95.0%	+18.8pp	0.24%
DOS trio (union, o-training)	5.1%	92.6%	+16.4pp	1.6%
trained clf — deployable point	0.0%	100.0%	n/a (1 fire)	0.0%
trained clf — F1-optimal point	100.0%	76.2%	+0.0pp	99.9%

The two classifier rows are the ends of one dial. At the *deployable* point (the only setting that does not constantly cry wolf) it fires once in 6,862 runs — effectively silent. At the *F1-optimal* point (best by the usual accuracy measures) it just predicts "everything failed": 100% fire, +0.0 lift, no skill. The widely-quoted "98% accuracy" lives at that second setting — the recall mirage of §7 in classifier's clothes. No dial position both fires usefully and stays trustworthy on an unseen task; our zero-training detectors sit at +16 to +19 lift at a deployable false-alarm rate.

Two further problems. The classifier reads the agent's own trajectory structure, so it risks the mirror-verifier trap (§1) — it can learn the tells a model stops emitting once trained against. And when we feed it the DOS detector flags as features, its top weights *are* those detectors (`dangling_fired`, `terminal_error_fired`, `tool_stream_fired`): it re-derives the cheap signals it was handed. The right place for such a model is not beside DOS as a rival but *under* it — a JUDGE-rung adjudicator (ORACLE → JUDGE → HUMAN) that rules only on the residue the deterministic detectors abstained on, fail-to-abstain, never the floor.

8.3. Arbiter neighbours

Two multi-agent arbitration systems sit nearby: Limen^[4] (advisory write-leases) and CodeCRDT^[5] (lock-free CRDT). Both compare to DOS's `arbitrate()` call, not its detector, and neither can run on this benchmark: a frozen single-agent trace has no live write to mediate. (The arbiter axis is not unmeasured in this paper —

correct for a single agent and we make no claim against it — only against the dismissal it smuggles in, that single-agent reliability is therefore trivial. Two parts of the loop are real work the phrase hides: the *check* must be one the agent cannot forge (a git-ancestry, registry-first ship oracle under a closed vocabulary, not "re-read the result," which falls into the same mirror-verifier trap), and the *loop* must sense fast enough to gate every turn (here the in-flight path needed a rewrite from a ~250 ms Python hook to a ~10 ms native one). Neither is a few lines of glue.

But the *headline* claim — the place a re-run wrapper provably cannot follow — is the concurrent one, and we concede the stronger objection first. If the shared store offered a transactional primitive (a serializable transaction, a compare-and-swap, a conditional write), a wrapper around even a lone agent could fuse check and write into one atomic step and close the race with no referee. Our claim is scoped to the regime that holds for agents in the wild: they act through high-level tool APIs (cancel a reservation, file a ticket) that expose only unconditional writes, so no such primitive is available to wrap. There the TOCTOU argument bites — A's read and write are separate non-atomic calls, so no number of re-runs makes two calls one. The number settles it: §2.5's arbiter prevents all 6 live would-be clobbers across its 8 racing pairs and a re-run wrapper prevents none, because each agent's own check passed. (The arbiter is a lock service supplying the serialization the store does not, lifted *outside* any single agent — which is why it works where a wrapper on A cannot.)

The honest converse is our own boundary. At the single easy hop where a wrapper *does* apply, handing the verdict back in-loop is mildly *harmful* (ΔB, Appendix D) — so we do not rest the headline payoff on the single-agent case. The concurrency case without a transactional store is where the entire *incremental* payoff over a wrapper lives.

8.2. Trained classifiers

The nearest neighbour is a *trained* classifier: a model fitted on a labelled set of past failures, which then predicts whether a new run failed (arXiv 2511.04032^[3], 2025, about 98% accuracy on its own data). On paper that beats our detectors by a wide margin. The catch is what the number costs, and where that 98% actually lives.

A classifier needs a labelled set of failures from the same kind of task, and a held-out slice to score it honestly. That labelled set is exactly the thing you do not have on a task you have never seen, which is the whole problem. Run the classifier *fairly* on this corpus and two things happen. Its best-balanced operating point just predicts "everything failed": on a benchmark where 76.2% of runs fail, that scores well by accuracy but buys no real skill, and this is where the 98% headline lives (the recall mirage from §7). Its only deployable operating point — the one that almost never cries wolf — fires exactly once. So at a false-alarm rate you could ship, it does nothing.

§2.5 exercises it live and counts $J = 6$; neither neighbour reports a comparable prevented-corruption count.) On this frozen target DOS's arbiter likewise has nothing to mediate, and its enforcement point — a pre-tool `permissionDecision: deny` hook an agent runtime such as Claude Code honours before a tool runs — is never exercised by a single-agent benchmark. The detector axis of this study and the arbiter axis of these neighbours do not share a scoreboard.

9. Limitations

- **In-loop: DETECT, not FIX — no task-lift number on Toolathlon.** Every Toolathlon figure is detection on frozen runs; the one validated *in-loop* action is give-up (§4), which repairs nothing. The *out-of-loop* result (§2, $J = 10$ across two models / 120 tasks) is a measured *existence* result at an 8.3% rate, not a calibrated population rate, and on a different benchmark (tau2-bench).
- **The recall ceiling is hard and tightens on the frontier.** Union recall is 6.18%–7.42%, so ~93% of failures stay silent, and fire-rate falls toward zero on the strongest models.
- **Four syscalls are null by construction here.** Toolathlon runs one agent on isolated state with no git or WAL, so `arbitrate`, `verify`, `liveness`, and `resume` have no job; only the in-flight detectors do.
- **The give-up token-saving magnitude is single-corpus, single-model.** The gate never wrongly halts a winner where it fires (specificity, across 22

models and two benchmarks); how *often* it fires, and so how much it saves, is benchmark-dependent.

10. Conclusion

Hold one thing fixed — a byte-clean verdict the agent cannot forge, scored by a third-party oracle DOS does not own — and the result is about *where* to spend it. *In-loop*, every active fix (warn, rewind, inject the environment’s own correction) was flat or net-negative, because injecting a turn perturbs runs that would have passed; the one in-loop action that helped was the negative one, give up correctly (**zero of 1,634 winners halted across 22 models**, ~11% of tokens saved). *Out-of-loop*, the same verdict pays on both ways a fleet fails — a write-admission gate blocked **10 over-claims** at an identical 8.3% rate across two models (so the signal does not fade at the stronger tier), and an arbiter prevented **all 6 would-be clobbers across 8 racing pairs** — both off a byte no agent wrote, neither touching the judged agent. The decisive move is not a better detector but a better *consumer*: detect cheaply in-loop, stop the doomed run, and spend the verdict out-of-loop — with the W3 residue (a goal with no world footprint) still beyond any witness short of a human.

10.1. Directions

The result generalizes along the axis that gave it: *where* a non-forgeable verdict is spent, and *what witness* it can read. Four directions follow from the honest edges above, each a measurement we know how to run. **Close the RLVR loop to a weight update.** §2.6 measures the label purge (poison reward labels removed off ground truth); the next rung is the trained-behaviour delta — fine-tune two models on the poison and the witness-cleaned reward set and measure whether the second over-claims less. The pipeline is built and the proxy separates (+60pp over a control that over-claims 0/4); only the weight update is unmeasured, and that is the one number a training lab would want first. **Raise the witness floor.** About 38% of frontier goals reach *no* sound witness at any rung (§3); the open question is how far a richer, still-non-forgeable witness (a structured state-diff, an environment-authored trace, a second independent grader) can push that floor down before the irreducible human-seed residue. **Cascade depth as a calibrated curve, not an existence shape.** Appendix D shows the payoff compounds F^D across a branching fleet, but with an injected root and per-node re-inheritance; a fully serial chain whose nodes *write* on the poison turns the shape into a calibrated constant. **Push the referee from advisory to enforcing, carefully.** Everything here is advisory — it proposes, a human or a

downstream system decides — which is the right default; the controlled question is where an out-of-loop *enforcement* point (a pre-write deny the runtime honours) can be trusted to act without a human in the loop, and where it cannot. The throughline is a small trusted referee that lets a fleet of fallible, self-narrating agents — and the loop that trains them — be scaled without scaling the trust placed in any one of them.

Reproducibility

Every number in the Toolathlon study (the detectors and their ceiling, §5–6; the give-up specificity, §4; and the naive baselines, §7) recomputes offline, with no network or model calls, from the durable per-run join `replay_all_rows.csv` (7,116 records, content fingerprint `sha256:1a55e8d8e2d4` — computed line-ending-normalized, CRLF→LF, so a checkout on Windows whose raw `sha256sum` differs is *not* seeing drift; `additivity.py:rows_fingerprint` is the canonical computation). The additivity single-source-of-truth re-asserts the headline claims and exits non-zero if a regenerated file drifts from them, so a claim cannot silently part from its data; the give-up cross-benchmark numbers and figure regenerate from their own script. The one exception is §2: it reports a *live, paid* run (tau2-bench through Gemini; single-model run \$0.89, cross-model run \$4.77), so its numbers are a fixed read-off of that run, not an offline recompute. They are transcribed verbatim into the figure’s single source of truth (`paper/_VERIFIED_FACTS_228_2026-06-08.md` and `_VERIFIED_FACTS_232_2026-06-08.md`), and the run is resumable from a per-task cache so it never re-spends; re-running it requires a Gemini API key and reproduces an *existence* result (the gate blocks real over-claims), not the exact count, since the live rollouts vary.

```
# 1. regenerate the durable rows from the frozen trajectories (offline once
_data/ is cached):
python -m benchmark.toolathlon.run_replay --all --no-download --by-model \
--rows-out benchmark/toolathlon/_results/replay_all_rows

# 2. recompute + re-assert the detector / additivity / union claims (exits
non-zero on drift):
python -m benchmark.toolathlon.additivity --check

# 3. regenerate the give-up-correctly numbers and the cross-benchmark
figure:
python benchmark/giveup_cross_benchmark.py

# 4. redraw every figure from the same single-source-of-truth numbers:
python -m benchmark.toolathlon.viz
```

Artifact. The replay harness, the per-run data, the figures, and the single-source-of-truth scripts that compute every number in this paper are released together, under `benchmark/toolathlon/` and `benchmark/` in the project repository. The Toolathlon benchmark and its `Toolathlon-Trajectories` dataset (CC-BY-4.0) are the work of the HKUST-NLP authors; we replay them here as an external, third-party-scored test target, and they are not a contribution of this paper.

Appendix A. DOS in plain words — the Vaseline test

The body defines its terms as it goes. This appendix is for a reader who wants the one idea underneath all of them — *do not trust the narrator* — in the plainest language, plus two reference tables: a key to every symbol (Table A1) and a plain-language glossary (Table A2). It is self-contained, built around one running picture: a smear of Vaseline on a mirror.

A.1 The one idea, and the Vaseline picture

An AI agent doing a long task also authors the story about it — the actions *and* the closing summary, and the summary is the problem. It can end with a confident “*All customers have now received their surveys!*” while the inbox shows zero emails sent: nothing crashed, nothing in the transcript contradicts the happy ending. This is a **silent failure**, a wrong result in the costume of a right one (on

the hard benchmark here three runs in four fail, and nearly every failure is this quiet kind). So the obvious move — read what the agent says about itself — is the **mirror-verifier** trap: you measure the *story*, not the *world*, and training against it just teaches the agent to stop saying the give-away phrase.

The picture: the real state of the world — the inbox, the database row, the file — is a mirror, and the agent’s narration is a **smear of Vaseline** wiped across it. Vaseline changes only what you *see*, and the smear re-applies for free (the agent can always produce a smoother sentence) — that is what *forgeable* means. The fix is to read only what the narration cannot coat: the **tool results the environment sent back** (an error envelope, the identical “pending” returned three times, the absent row) and a **third-party judge** that re-reads the live world. The agent authored none of those bytes. A check that reads *only* bytes the judged agent did not author is **byte-clean**; the test is one question — *could the agent have written these bytes?* If yes, Vaseline; if no, ground truth.

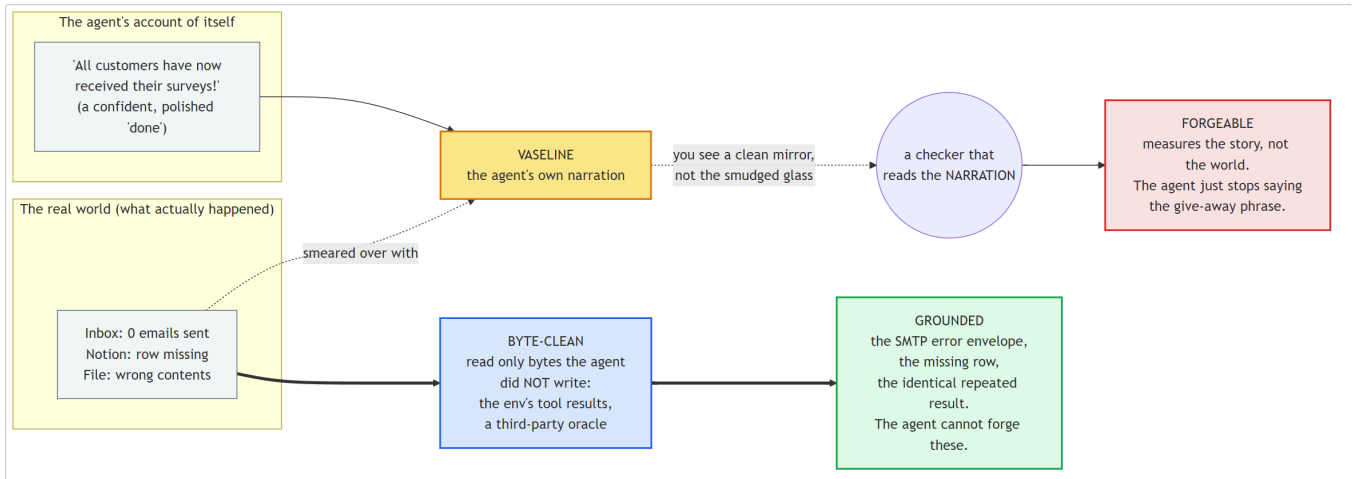


Figure A1. The Vaseline test. The agent's narration is a smear over the real state of the world: it changes only what a careless checker sees. Reading *through* the smear (the agent's words) is **forgeable**; reading *around* it (the environment's tool results, a third-party oracle) is **byte-clean** and grounded. Every DOS check lives on the bottom path.

A.2 What DOS actually is — a referee made of "no"

DOS — **Dispatch Operating System** — sits between many AI agents and the one shared world they are all changing, the way a computer's OS sits between many programs and one machine, and plays **referee**. Its motto is blunt: *the*

kernel is the part that does not believe the agents. A referee mostly says one word, **no**, and DOS is a handful of careful ways to say it — each the same move, a refusal backed by evidence the agent could not forge, pointed at a different question (the map below).

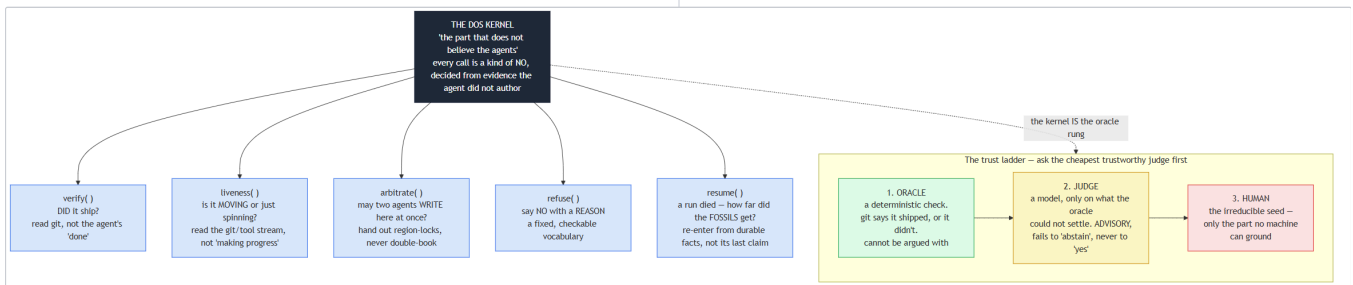


Figure A2. The DOS kernel is a small set of grounded refusals — `verify`, `liveness`, `arbitrate`, `refuse`, `resume` — each answering one question from bytes the agent did not author (git history, not the agent's "done"). Below, the **trust ladder**: a deterministic **oracle** settles what it can; a model-based **judge** rules only on the leftovers, as advice that fails safe to "abstain"; a **human** is the last resort for what no machine can ground.

Two features keep the referee honest. **The oracle goes first and cannot be argued with** — "did this phase ship?" is answered from git history, a record the agent cannot rewrite to flatter itself. **The model-judge is on a short leash** — used only on what the oracle could not settle, only as advice (it never blocks alone), and when unsure it must say "abstain," never "looks fine," because a model checking a model is still two things on the smeared side of the glass.

A.3 The three detectors, lift vs. recall, and the punchline

This paper measures three of those refusals as **detectors** (defined in Table A2), each firing on a byte-clean fingerprint of failure: `dangling_intent`, `tool_stream`, `terminal_error`. They produce one triumphant number and one dismal one, both true: **when they fire they are right 88–98% of the time** (a *lift* of up to +22 points above the 76% base fail rate), but they **catch only ~2% of failures** each. Picture a smoke alarm that is 98% accurate but goes off in two of every hundred

real fires: run when it sounds, but keep your other alarms. And it sharpens on the frontier — the strongest models fail *quietly* (a polished, wrong summary, no trace), so the fire-rate drifts toward zero exactly where the leaderboard lives. Quality carries; coverage shrinks.

The punchline is **detect, don't fix**. In a separate live program every *active* fix came up empty or worse — **warn** (flat, +0.2 points), **rewind** (zero conversions), **cure** with the environment's own correction (net-negative). The harm is the intervention *existing in the loop at all*, not its content. The one survivor is the **negative** action, **give up correctly**: it fires on a repeated *environment-authored error* and, across two benchmarks and 1,600+ winners, halted **zero** winners while saving ~11% of compute. (A naive version that halts on any *repeat* kills six winners — one model harmlessly polling. Repeating is normal; repeating an error is the doomed signal.)

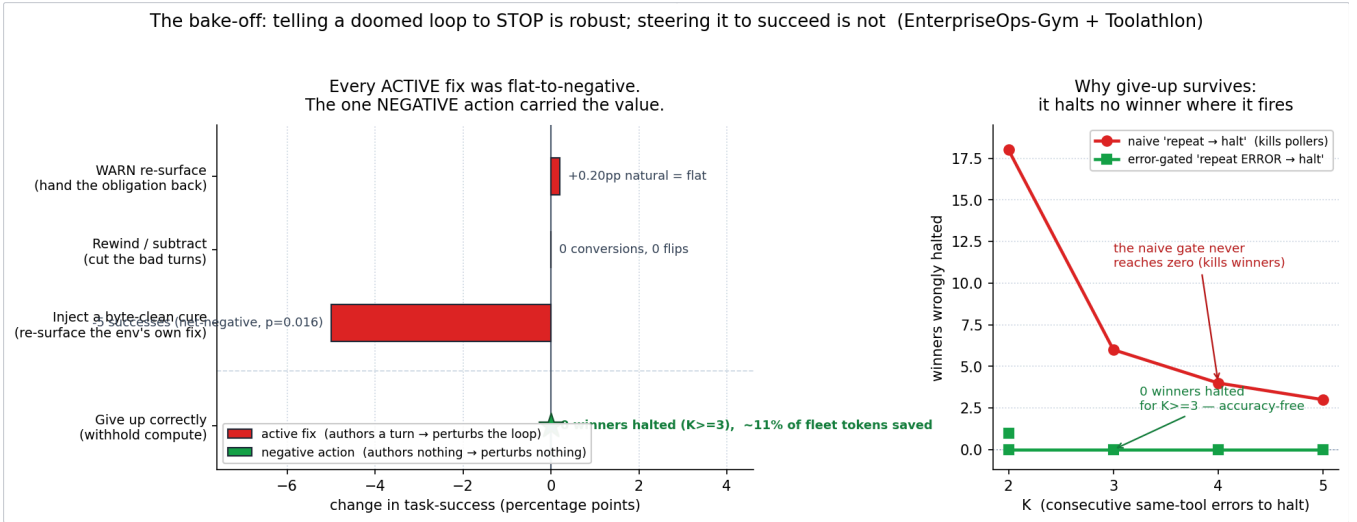


Figure A3. The bake-off: steering a doomed loop fails, stopping it works. **Left:** every *active* fix runs flat (warn) to nothing (rewind) to negative (the injected cure), because each authors a turn and a turn perturbs a fragile run; only the *negative* action (green star), give up, authors nothing. **Right:** the error-gated halt wrongly stops **zero** winners at $K \geq 3$ while a naive halt-on-any-repeat never reaches zero. (From EnterpriseOps-Gym and Toolathlon, kept apart from the main detection tables.)

Telling a doomed loop to stop is robust; steering it to succeed is not. So the substrate detects, withholds compute when it is sure, and hands the rest to a human, rather than pretending it can fix what it can only see.

A.4 Where the substrate ends and a verifier begins

The failure that beats every detector here has no fingerprint at all: the agent runs cleanly, hits no error, contradicts nothing, and ends with a confident, polished, *wrong* summary, because it never re-read the state it claims to have changed. No byte already in the transcript reveals it; the only honest catch is to **go re-read the world** after the run. That is the job of the benchmark's own final judge, and

the line this paper draws on purpose: an in-flight byte-clean substrate reads the evidence the environment already wrote; a **final-state verifier** re-reads the world to settle what no in-trace evidence can. The gap is not a flaw to hide — it is the boundary, where reading around the Vaseline runs out and going behind the mirror begins.

A.5 Notation — the symbols, decoded

Every symbol in one place. The body leans on **J** on nearly every page of the headline result and glosses it only briefly at first use; this is its full key.

Table A1. Every symbol and short code in this paper.

Symbol	Reads as	What it means, plainly
J	the prevention count	An integer count of <i>real corruptions the referee caught and blocked at the root</i> — phantom writes a peer would have inherited, or overwrites a race would have caused. Not a rate, and <i>not</i> a downstream-outcome delta (that quantity is ΔB , Appendix D). " $J = 10$ " = ten real failures stopped; the whole §2 result is in J. J is experiment-local — a blocked over-claim and a prevented clobber are different events — so compare J within one experiment, never across them.
K	the halt threshold	How many times the <i>same</i> tool must error <i>in a row</i> before the give-up gate proposes a stop. We use $K = 3$ (latest still failing) — the smallest value that never killed a winner across 22 models.
pp	percentage points	The plain <i>difference</i> between two percentages: 76% to 92% is "+16 pp", not "+21%" (the relative change). Lift is always in pp.
db_match	"database matches"	The unforgeable witness in §2: the benchmark hashes the task's database and compares it to the <i>gold</i> hash. True = world correct; False = not; absent = no database to check. The agent writes none of these bytes.
Wo–W3	witness rungs	Four grades of evidence (§3): Wo the agent's narration (forgeable); W1 the environment acknowledged a write; W2 the change <i>persisted</i> (DOS's git rung — <i>presence</i>); W3 a full comparison to the correct answer (<i>correctness</i> , what db_match does). Only W3 settles whether the work was <i>right</i> .
tp / fp / fn	true-positive / false-positive / false-negative	A fire is a <i>true-positive</i> (run really failed) or <i>false-positive</i> (run passed — the costly false alarm); silence on a failing run is a <i>false-negative</i> (the missed silent failure). With true-negative they form the confusion grid.
Wilson-95	"Wilson 95% upper bound"	A conservative ceiling on a rate measured as zero: "Wilson-95 upper 0.0023" means the <i>true</i> rate is almost certainly below 0.23% — a guard against reading "0 out of N" as stronger than the sample supports.

A.6 Glossary — every term, in plain words

Grouped by the kind of thing each is; every entry is self-contained.

Table A2. The terms used in this paper, in plain language, grouped by topic.

Term	What it means, plainly
The one idea, and its vocabulary	
silent failure	A run that ends "successfully" by its own account but left the world wrong — no crash, no contradiction, just a confident false "done." Three runs in four fail on the hard benchmark here, nearly all of them this quietly.
byte-clean	A check that reads <i>only</i> bytes the judged agent did not write (the environment's tool results, a third-party judge) — the thing its narration cannot smear. The single rule under everything DOS does.
byte-author	Whoever produced the bytes a check reads. A check is trustworthy only when its byte-author is <i>not</i> the agent being judged.
forgeable	The opposite of byte-clean: a signal the agent <i>can</i> author and therefore wipe away (its narration is forgeable; an error envelope the mail server wrote is not).
the Vaseline test	The one-question check: <i>could the agent have written these bytes?</i> If yes, the smear (forgeable); if no, the mirror (byte-clean).
mirror-verifier trap	Checking an agent by asking it how it looks — measuring consistency (does the story hang together?), not truth (does it match the world?).
consistency ≠ grounding	Re-deriving an author's <i>own</i> bytes proves consistency; checking against a <i>different</i> author's bytes proves grounding. Only grounding crosses the trust floor.
The referee — DOS and its parts	
DOS	Dispatch Operating System: the small trusted referee between many agents and the one shared world they change; it believes none of their self-reports.
kernel	The small trusted core — the referee itself — as opposed to the agents it watches. "The part that does not believe the agents."
oracle	A deterministic check that cannot be argued with (git history says a phase shipped, or it didn't). Always tried first.
third-party oracle	The benchmark's own pass/fail judge, authored by the benchmark not DOS — a verdict DOS cannot fake. Every detector here is scored against one.
judge	A model-based check, used only on what the oracle could not settle. Advisory; says "abstain" when unsure; never blocks on its own.
ORACLE → JUDGE → HUMAN	The trust ladder: a deterministic oracle settles what it can, a model-judge rules only on the leftovers as advice that fails safe to "abstain", a human is the last resort.
abstain	The safe non-answer: when evidence is missing, a check says "abstain" rather than guess. It can refuse on proof but never <i>affirms</i> without it; abstain always means admit.
advisory	A check that <i>reports</i> a verdict but does not <i>enforce</i> it. Everything measured here is advisory; nothing kills a run on its own.
detector / fire	A check that "fires" on a byte-clean fingerprint of failure; fire-rate = how often it flags, as a fraction of all runs.
structured error envelope	The specific <i>shape</i> of an environment-authored failure (a crash, a 5xx, a permission-denied) that <code>terminal_error</code> matches — not merely the <i>word</i> "error" in a result. Matching the shape keeps false alarms near zero.
Where you spend the verdict	
in-loop	Spending a verdict <i>back on the agent that earned it</i> (warn, rewind, correct). Flat-to-harmful here, because adding any turn disturbs a run that might have passed.
out-of-loop	Spending the <i>same</i> verdict on <i>anyone but</i> that agent — a peer, the operator, a training set. Where it pays: correcting a peer's input injects nothing back.
claim (failure)	The first way a fleet fails: one agent announces a write that did not land, and a peer inherits that false "done" as truth.
race (failure)	The second way a fleet fails: two agents touch the same state at once and one silently overwrites the other. No one lied; the state is wrong anyway.
write-admission gate	The out-of-loop referee for a <i>claim</i> : it checks A's announcement against a byte A could not author before a peer inherits it — admitting a real write, blocking a phantom.
over-claim / phantom write	An agent announcing a successful change the world's own record says did not land. What the write-admission gate catches.
clobber / lost update	What a race causes: a second agent's write lands on top of a first's, erasing it, because the second never saw the first. What the arbiter prevents.
lease / region / lane	A <i>region</i> is a named patch of shared state (e.g. <code>reservations/<id></code>); a <i>lease</i> is a temporary exclusive hold. Two agents wanting overlapping regions cannot both hold one.
arbiter / arbitrate	The referee for a <i>race</i> : it hands out region leases and refuses a second agent's lease while a first holds an overlapping one, so the writes <i>serialize</i> .
serialize	To force two would-be-simultaneous writes into a strict order, so the second acts on the first's result instead of clobbering it.
half-plane	One of the two distinct ways a fleet fails (a claim and a race); the paper's symmetry is one referee covering both off the same witness.
existence result	A claim stated no wider than "this happens and we caught it N times," not "X% everywhere." §2 proves the gate catches real over-claims live, not that 8.3% is universal.
The give-up result	
DETECT vs FIX	Spotting a failure vs repairing it. This paper measures DETECT only. The one FIX-adjacent action that helped was the negative one — give up correctly.
give up correctly	Stop a doomed run and re-spend its token budget elsewhere. Authors no new turn, so it perturbs nothing — the only in-loop action that paid off.
false-abandon	The give-up gate's one costly mistake: halting a run that <i>would</i> have succeeded. The number that has to be zero — and is, at K = 3.
specificity vs sensitivity	<i>Specificity</i> : when it fires, it never halts a winner (the half we have, across 22 models). <i>Sensitivity</i> : it fires often enough to catch most doomed runs (the half we do not — quiet on silent failures).
circuit breaker / SPRT / CUSUM	Three classic "stop after repeated trouble" mechanisms — a circuit breaker (trip a failing component after too many faults), SPRT (decide as evidence arrives), CUSUM (watch a running tally for a change). The give-up gate is a corner of each; only its byte-clean <i>grounding</i> and measured threshold are new.
Measuring a detector	
precision / recall / lift	When it fires, how often it is right (ours: 88–98%); of all failures, how many it caught (ours: ~2% each, ~6% combined); and lift = precision – the 76.2% base fail rate, the "better than guessing" part we read throughout instead of raw precision.
false-alarm rate	Of all the runs that actually <i>passed</i> , how often the detector wrongly flagged one — the costly mistake (cry wolf and an operator stops listening). Ours: near zero.
net-new / additivity	A catch a detector makes that the others did <i>not</i> , so adding it raises coverage instead of re-finding the same failures. <code>terminal_error</code> is 99% net-new.
union (pair / trio)	Treating a <i>set</i> of detectors as one (caught if <i>any</i> fires): the <i>pair</i> is the first two, the <i>trio</i> all three.
operating point	A classifier's dial setting trading eagerness-to-fire against false alarms. A trained classifier has no setting that both fires usefully <i>and</i> stays trustworthy on a task it was not trained for.
Systems and benchmarks named in the body	
Toolathlon	The hard benchmark behind the offline replay study (every section except §2): long tasks over 600+ real tools across 32 apps, with its own pass/fail oracle re-reading the real app state. DOS replays its published recordings.
trajectory / replay	A <i>trajectory</i> is one agent's recorded transcript on one task; <i>replay</i> is running the detectors over those frozen recordings offline (§0) — which is why the study measures DETECT, never FIX.
tau2-bench	The live benchmark in §2 (airline/retail customer service), the right target because it ships <code>db_match</code> — the database-hash witness the agent cannot author.
EnterpriseOps-Gym	The live benchmark for the \$4.3 bake-off (a ServiceNow gym with hidden database checks), used because an active fix can only be measured by replaying it into a <i>live</i> agent.
Gemini (flash / pro), grok-4	The models run live in §2 (<code>gemini-2.5-flash</code> mid, <code>gemini-2.5-pro</code> stronger) and the one that exposed the naive gate's flaw by harmlessly polling (grok-4).

Term	What it means, plainly	
SMTP / 5xx / API error	Ordinary failure shapes the environment reports (a mail-server "no", a server-side HTTP error, a transient call failure) — all environment-authored bytes the agent cannot talk away.	
The boundary and beyond		
final-state verifier	A check that re-reads the live world after a run to catch	the confident-but-wrong ending no in-trace evidence reveals. The boundary where an in-flight detector ends and §2's out-of-loop re-read begins.
RLVR / poison reward label	RLVR = Reinforcement Learning from Verifiable Rewards. A <i>poison reward label</i> is an over-claim a self-judged loop banks as a <i>win</i> , teaching the model to over-claim more; §2.6's gate purges them off ground truth.	

The companion engineering write-up for this appendix — with the same diagrams and the "why doesn't the lift carry over?" walk-through — lives at `benchmark/toolathlon/EXPLAINER.md` and `GLOSSARY.md` in the DOS repository.

Appendix B. How it works — from a forgeable claim to a grounded refusal

Appendix A said *what* DOS believes and *where* a verdict pays. This appendix says *how* a verdict is computed — the machine that turns a forgeable sentence into a grounded "no" — and shows the two live consumers from §2 are the *same* pipeline pointed at the two ways a fleet fails. The reference points are real: the gate is `benchmark/agentprocessbench/writeadmit/gate.py`, the referee-between-agents is `coord_loop.py`, and the floor both stand on is `dos.effect_witness`.

B.1 The pipeline: four steps, and only one of them decides

Every DOS verdict — the write-admission gate, the race arbiter, every detector here — runs the same four-step pipeline. What does the work is not any one clever step but the strict separation between them.

1. **Gather evidence — at the boundary, never inside the decision.** All reading from the messy world happens here and only here (the database hash, the tool-result stream, the git log); it judges nothing, just collects bytes and notes *who wrote each one*.
2. **Tag each byte with its author.** Every piece of evidence is labelled by which rung of trust it came from — agent, environment, or third party. This label decides everything downstream (B.2).
3. **Classify — a pure function, no I/O.** One function takes the tagged evidence and a policy and returns a verdict, reading no files and asking no model, so the same evidence returns the same verdict forever. The only step that decides.
4. **Return a typed refusal — from a closed list.** One value from a fixed vocabulary (admit / block / abstain, with a named reason), so a consumer branches without parsing prose and a human can check every answer was accounted for.

Splitting reading from deciding is the reason a courtroom separates gathering testimony from rendering a verdict: a decision that re-reads the world while deciding can be swayed by what it re-reads. By the time the pure classifier runs, the evidence is fixed and labelled and the agent has no further move. *I/O at the boundary, a pure verdict at the core* is the one structural rule the kernel obeys.

B.2 The accountability ladder — why the agent's own words can never tip the scale

Step 2 is where byte-clean turns from a slogan into a working mechanism. Each piece of evidence carries a label naming its author, and the labels are ordered by how hard they are to forge:

- **AGENT-AUTHORED** — the agent wrote it (its final message, its self-report). This is the Vaseline: admitted into evidence but marked, and that mark keeps it from deciding anything.

- **OS-RECORDED** — the environment wrote it (the database hash, the mail server's error envelope, the git commit). The agent triggered the action but did not author the byte.

- **THIRD-PARTY** — an independent grader wrote it (Toolathlon's `evaluation/main.py`, tau2's database-hash check). Authored by neither the agent nor DOS.

The rule that makes the floor unforgeable: **only an OS-RECORDED or THIRD-PARTY witness may set the "refuted" bit**. An AGENT-AUTHORED byte can be *recorded* (the verdict notes the agent *claimed* success) but can never *confirm* or *refute*. So a policy that writes itself a glowing receipt moves nothing — its bytes land on the forgeable floor, treated as *no evidence either way*; only a byte from a higher rung flips a write from "admit" to "block". In code this is the `believe_under_floor` discipline: a read-back on the AGENT-AUTHORED rung yields *unwitnessed*, never *refuted*.

The agent's words are admitted as *testimony*, never as *proof*: only a byte the agent did not author can decide the case. This is the A.2 trust ladder (ORACLE → JUDGE → HUMAN) seen from the evidence side: an accountable witness ranks above the agent's narration, and a missing witness fails safe to "admit" — trust the unforgeable thing first, and when it is silent, do the cautious thing, never the flattering one.

B.3 The two consumers as one pipeline, two witness-moments

The write-admission gate of §2 is the four steps turning. **Gather** A's final message (the claim) and the tau2 evaluator's `db_match` result. **Tag** the message AGENT-AUTHORED and `db_match` OS-RECORDED (True an attestation, False a refutation, None no witness). **Classify**: does A confidently claim a write (a closed write-verb $\cap$ mutating-tool test, not a keyword sniff) *and* does an accountable witness refute it? Block only when both. **Return** admit / block / abstain. The structure buys three properties: it **cannot be talked into admitting** (only an OS-RECORDED byte sets "refuted," so a fake receipt changes nothing — the gate can only block *more* under a real witness, never less); it **never blocks correct work** (a database-confirmed write is attested, so all nine honest writes were admitted and only the five refuted ones blocked); and **blocking B's input writes nothing into A's loop**, so the in-loop harm of §4.3 is structurally absent — the entire reason the consumer is B, not A.

The race arbiter of §2.5 is the *same* pipeline with one substitution: the witness is read *before* the write, not after. A reservation is a region `reservations/<id>`; the pure call `arbitrate(region, live_leases)` refuses A2's lease on a region A1 holds (the same disjointness verdict the live `dos lease-lane` uses, no model, no I/O), so A2 — re-run after A1 commits — sees R already cancelled and does nothing. Its scoring witness is the same airline database hash, and a clobber is counted only when *directional* (the discipline that took the headline from 7 to an honest 6, §2.5). The arbiter's refusal is a refusal in the identical sense the gate's block is: a typed "no" from a pure verdict over evidence the agent cannot forge. One reads the witness after the claim, the other before the write. Two moments, one machine.

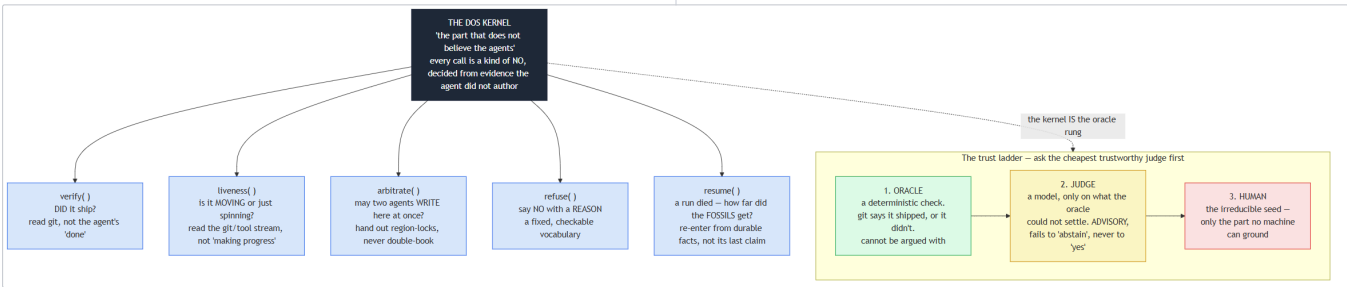


Figure B1. The same kernel map as Appendix A, read as *machinery*: each call is the B.1 pipeline with a different evidence source at step 1 (`verify` reads git, `arbitrate` the live leases, the gate the database hash), the same author-tag at step 2, one pure classification at step 3, and the same closed-vocabulary "no" at step 4. The trust ladder beneath is the B.2 accountability ladder from the adjudicator's side.

B.4 Why the answer is a closed list, not a sentence

The fourth step — return a value from a fixed vocabulary — carries real weight. A

free-text verdict ("looks mostly fine, but maybe double-check the booking") cannot be acted on by a machine or audited by a human: you cannot enumerate the prose a model might emit, so you cannot prove you handled every case. A

verdict that is one of {admit, block, abstain} with a named reason can be branched on without parsing, logged without ambiguity, and — the part that matters for trust — *checked for completeness*. This is why "abstain" is a first-class member, not a shrug: a verdict allowed to say "I have no witness for this" never has to guess, and a kernel that never guesses is one whose every "block" you can believe. That closure also lets the same verdict be spent in two places without re-deciding — the gate's "block" and the arbiter's "refuse" are the same typed object a consumer routes (drop the write, hold the lease, surface it) but never re-litigates. *That* is the mechanism behind the paper's one sentence: "handed to anyone else" is nothing more than routing a closed, grounded value to a consumer who is not the one being judged.

The whole machine: read bytes at the boundary, tag each by its author, decide once with a pure function that only an unforgeable byte can sway, emit one value from a closed list — then route that value anywhere except back to the agent it judged.

The companion engineering write-up — the gate, the arbiter, and the `effect_witness` floor with their tests — lives under `benchmark/agentprocessbench/writeadmit/` and `src/dos/effect_witness.py` in the DOS repository; the kernel's own architecture contract is `CLAUDE.md` and `docs/ARCHITECTURE.md`.

Appendix C. How to reproduce — every number, command, and released item

This appendix lists every result the body claims, the command that regenerates it, the artifact that backs it, and what you need to run it — so a stranger with the repository and one API key can re-derive the numbers, or see exactly what stands between them and a gated one. The frozen results are **\$0 and deterministic** (a shipped CSV or corpus); the live results are **paid** (an external benchmark through a model API). The distinction is the methodological spine: a frozen replay gives a *rate*, only a live run gives an *intervention payoff*.

C.1 The reproduction principle: re-derive, never trust the cache

One rule governs every script below, aimed at our own pipeline: **every number is recomputed from the byte the agent could not author, never read back from a stored summary**. The frozen results recompute from a per-run CSV the benchmark's own grader wrote; the live results recompute each verdict from the agent's stored message *with the current extractor*, not the cached bit. This is not pedantry — in the cross-model run (§2.4) a naive sum of cached bits reported the stronger model at $J = 4$, and re-deriving from the stored message gave the true $J = 5$, recovering an over-claim the older extractor had missed. So each command ends in a fold that re-reads ground truth, and several ship a `--check` mode that exits non-zero on drift.

C.2 One-time setup

Everything runs from a single editable install of the kernel plus its benchmark extras. The kernel's own dependency set is PyYAML-only; the paid runs pull in the external benchmark and a model client.

Table C1. Setup, in order. Steps 1–2 suffice for the two \$0 results; steps 3–4 are needed only for the two live results.

#	Step	Command	Needed for
1	Clone + editable-install the kernel	<code>pip install -e .</code> (from the repo root)	everything
2	Confirm the suite is green	<code>python -m pytest -q</code>	everything
3	Clone + install tau2-bench (the live target; the PyPI <code>tau2</code> is an unrelated squatter)	<code>pip install -e ./tau2-bench</code> then <code>pip install audiop-lts on Python 3.13</code>	\$2 live runs
4	Put a model key in <code>.env</code> at the repo root	<code>GEMINI_API_KEY=...</code> (the runners load it from <code>.env</code> ; it is not read from the shell)	\$2 live runs

Two setup traps cost real time. `tau2-bench` transitively imports `audiop` (removed in Python 3.13) — install the `audiop-lts` backport or the import fails before any task runs. And a Gemini access-token key (`AQ.`-prefixed) *expires*; re-smoke it before any batch (`models?pageSize=1&key=...`), or a dead key silently zeroes the run and makes “we tried, it found nothing” look like a result.

C.3 The two \$0 results (frozen, deterministic, no key)

These read a shipped CSV and recompute the paper's numbers with no network, no model, no spend. They are the detector-quality story (§6 / Appendix A.3) and the give-up story (§4 / Appendix A.3). Each is its own single source of truth and ships a `--check` mode that fails loud on drift.

Table C2. The frozen results — reproduce with one command each, \$0.

Paper claim	Command	Backing artifact
Detector precision 88–98%, recall ~2%, the trio-additivity +30% recall gain (§6, Table 3)	<code>python -m benchmark.toolathlon.additivity (print) · --check (assert) · --emit (write the claims ledger)</code>	<code>benchmark/toolathlon/_result</code> — 7,116 frozen runs, 22 models × 11 + detector sig
Give-up halts 0 winners at $K \geq 3$, ~11% tokens saved; the naive variant kills 6 (§4, Table 2)	<code>python benchmark/giveup_cross_benchmark.py (tables + figure) · --no-fig</code>	the same <code>replay_all_rows.c</code> <code>benchmark/enterpriseops/liv</code>
The frozen 13.6% over-claim slice (§2.2, the rate the live run promotes)	<code>python -m benchmark.agentprocessbench.overclaim --check → consensus=34 (13.6%) , exit 0</code>	the 250 frozen <code>AgentProcessB</code> (<code>dataset.load(config</code>

The durable-rows CSV carries a fingerprint, and the paper records it: the rows this paper's numbers were drawn from fingerprint to `1a55e8d8e2d4` (`additivity.rows_fingerprint()`). The paper's own build step warns if the live CSV has drifted from that value, so a stale paper cannot quietly ship wrong provenance. If you regenerate the rows and the fingerprint moves, the numbers move with it — that is the point.

C.4 The live results (paid, gated behind a key)

These are the headline of the paper — the out-of-loop payoff, run live (§2). They drive a real agent through `tau2-bench`, read the environment's own database hash, and count an integer J of corruptions a referee actually prevented. Both the agent and the user-simulator are live models, so cost scales with dialogue length; the whole program in the paper ran for well under the budgets below. Every live-output directory is **gitignored on purpose** — the seed configs embed the API key — so the repository ships the *drivers* that regenerate the result, not the raw transcripts. The \$0 synthetic smoke that each driver ships proves the mechanism with hand-built tool-calls and no model; run it first, every time.

Table C3. The live results. Run the \$0 smoke first; the paid command reproduces the paper's J. The two write-admission tables and the Appendix D programs — same discipline, separate drivers and spend together cost under \$8 (\$0.89 + \$4.77 + \$1.50); the saturation row is the *same* write-admission program to completion, so its \$8.35 is cumulative (it contains the first two runs). The Appendix D programs (Table C4) are **Paper claim** **\$0 / keyless (mechanism)** **Gated run (mechanism)**

Paper claim	\$0 smoke (mechanism)	Paid run (the payoff)	Paper claim	\$0 / keyless (mechanism)	Gated run (mechanism)
Write-admission, single model: J = 5, 11.6% over-claim rate (§2.3, Figure 2)	python -m benchmark.agentprocessbench.writeadmit.live_loop with frozen_ab (no key) — J arithmetic proven	benchmark/agentprocessbench/writeadmit/_run_model.py --smoke --live -p --model 5 -flash <out_dir> 30 25	Witness + subject of 35 confident writes (66%), 25 of 31 (81%); AB = 4 flash / 0 pro (D.1)	python -m benchmark.agentdiff.live_loop (no key) prints frozen-witness summary; test_gate.py / test_live.py	same modu (resumable split caches ΔB runner; (dc
Cross-model: J = 10 over 120 clean tasks, 8.3% on both flash and pro (§2.4, Figure 3)	(same gate; the smoke above covers it)	run _run_model.py once for gemini/gemini-2.5-pro, then fold: python -m benchmark.agentprocessbench.writeadmit.aggregate_live	The frontier ladder: grow the fold with further re-fold (docs/228	python benchmark/fleet_horizon/forgo.py — authors the nine artifacts as real commits and runs the deterministic ladder keyless (the judge seat abstains); the run of record is results table5_ladder_20260609.json	DOS_LLM_JUI -p --model 5' seats t through th (dc
Saturation: J = 15 over 258 clean tasks, whole-distribution rate 5.8% (§2.6)	python -m benchmark.agentprocessbench.writeadmit.aggregate_live — re-folds every cached run directory and re-derives each verdict from the stored message, \$0 once the caches exist		foiled 7/8, presence 2/8, behavior 0/8; the judge is functional only given the OS test byte (D.2)		
Coordination: J = 6, arbiter serialized 8/8 pairs (§2.5, Figure 4). 6 is the corrected count: the directional-clobber definition (§2.5) drops one run-to-run-variance pair, taking the live headline from a reported 7 to an honest 6 — so do not expect a raw 7.	python benchmark/agentprocessbench/writeadmit/coord_loop.py --smoke — the arbiter-serialization invariant, no key	benchmark/agentprocessbench/writeadmit/coord_loop.py	The cascade: chain D→1 (1, 2, 3 at depths 2→4); tree F ^D (4, 8 at depths 2→3, F=2) (D.3)	python benchmark/agentprocessbench/writeadmit/cascade_loop.py --smoke — the synthetic state-cascade mechanism, no key	--live -chain) and -depth 3 -tree), \$2.50 do

Two model-specific details silently zero a run if missed. First, the cross-model run needs a **per-tier reasoning setting**: the flag that suppresses a flash-only crash on long dialogues is fatal to the thinking-only pro tier (“*Budget 0 is invalid*”) — a pro attempt that re-used the flash flag errored on all 60 tasks and reported a false J = 0 (a regression test test_model_args.py now pins it). Second, the coordination run must give each task the reservation's **real owner** as the user identity, or a correct agent rightly refuses to touch a stranger's booking and the designed conflict never fires. Both are harness details that make “we tried, it found nothing” an artifact rather than a result.

The complete, append-only run journals for the live program — every checkpoint, every trap, every dollar — live next to the drivers and are the most faithful record of how the numbers were produced: benchmark/agentprocessbench/writeadmit/RUN_PLAN_live.md (the write-admission run, J = 5 → the design notes that became docs/228), and the matching design notes docs/232 (cross-model, J = 10) and docs/233 (coordination, J = 6).

The Appendix D programs reproduce the same way, from their own drivers and journals (Table C4). Each follows the Table C3 pattern: a keyless command that proves the mechanism, then a gated command that reproduces the live number.

C.5 Released items

The full inventory — the kernel (src/dos/; distribution dos-kernel, import/CLI dos), the seven-benchmark suite (python -m benchmark.run list prints each one's prerequisites and cost; the three the paper draws on are toolathlon, agentprocessbench, and enterpriseops), the design notes, and the paper's own reproducible build (python paper/build.py) — ships in the project repository. The dependency arrow is one-way: the benchmarks and the paper import dos / call the dos CLI; nothing under src/dos/ depends on them. The write-admission floor both live consumers stand on is src/dos/effect_witness.py (the believe_under_floor discipline of Appendix B.2); the build is single-sourced from paper/meta.py, with every figure regenerated from the run data.

The shortest path from a clean clone to the numbers: three commands recompute the frozen rates from shipped data at \$0 (Table C2); the live programs (Tables C3–C4) drive real agents through tau2-bench and Agent-Diff off a single key to recompute the intervention payoffs — and every one re-derives its number from the byte the agent could not author, never from a cached summary.

Appendix D. Extended out-of-loop results

The body (§2) carries the two headline live results — the write-admission gate (J = 10) and the race arbiter (J = 6). This appendix holds three further results that *strengthen* the existence claim without being the headline: a richer witness and the first measurement of what one propagated phantom costs (D.1); a frontier test that puts a frontier model in the verifier's own seat against a ladder of non-forgeable rungs (D.2); and the compounding payoff across a branching fleet (D.3). All three read the same kind of byte the agent cannot author, and each carries its evidence-strength label in the same breath as its number.

A.1. D.1 A richer witness, and the first measurement of what propagation costs

tau2's witness is a single bit, so a reader might ask whether the result is an artifact of that coarse signal. We re-aimed the *identical* gate at a second benchmark with a much richer witness: **Agent-Diff** (a write-heavy enterprise-API benchmark over Slack, Box, Linear, and Google Calendar), whose environment computes a full *structured state-diff* from database snapshots — the exact rows inserted, updated, deleted — and grades against a multi-assertion gold specification. Driving an agent live through Gemini on the 45-task test split, the **detection result holds across the capability step a second time**: flash over-claims on

23 of its 35 confident writes (66%), pro on 25 of 31 (81% — *more*, not less), and the gate blocks every one. The over-claims are partly *model-specific* (14 tasks over-claim on both models, the rest on only one), so the gate catches the *deployed* model's blind spots, not a fixed set of impossible tasks. Detection generalizes off the coarse `db_match` bit to a structured diff and across two tiers.

Agent-Diff also lets us measure the thing J names but does not itself quantify: **what does letting a phantom write propagate actually cost a downstream agent?** We build the peer B handoff explicitly. For each blocked over-claim we run a second agent twice from the *same* fresh, correct environment, changing only what B is told it inherited: under **believe** B receives A's claim verbatim; under **adjudicate** B receives the gate's env-verified correction ("the prior write did not land — re-verify before acting"). The measured quantity is

$$\Delta B = \text{success}(B \mid \text{adjudicate}) - \text{success}(B \mid \text{believe})$$

over the over-claim slice. On flash, $\Delta B = +4$ (four flips, zero reverses): telling the peer the truth turned four phantom-induced failures into successes and never made one worse. On pro, $\Delta B = 0$ (two flips cancelled by two reverses; one reverse traced to a real gate bug — a blanket "treat the records as unchanged" over-corrected a *partial* over-claim in which one of two writes had landed. We fixed the gate for future runs but report the as-run number).

We lead with J, not ΔB , because they are a product and a lower bound on one of its terms, not two estimates of one thing. J counts the lies stopped at the root; ΔB measures the cost of letting *one* lie propagate *one* hop to the most *forgiving* consumer imaginable — an LLM peer that re-runs the task and can sometimes heal the missing write for free. So ΔB is a pessimistic floor: the gate's real value is **blocks $\times$ depth $\times$ per-hop damage**, and ΔB pins depth and consumer to their

minimum. A consumer that does *not* re-check (a reward labeler, an auto-merge, a dashboard) recovers nothing. The *depth* term is what D.3 measures.

A.2. D.2 The frontier test: a ladder of rungs a frontier model cannot all climb

The cross-model results answer "does a *stronger agent* over-claim less" — it does not. The sharper version of the objection is that **the latest model is smart enough that you no longer need a referee** — either it stops over-claiming, or, as a judge, it sees through forgeries an older checker missed. We test the decisive second half: the referee here has two implementations, the deterministic floor that reads ground truth or a model asked to *judge* the same claim, so we put a frontier model in the judge seat and raced it against the floor. The model is **Table 5** (the newest at the time of writing), reached the way an operator would — straight through the Claude Code command line (`claude -p`), the model named only in an environment variable, *not a line of model-specific code* in the harness.

The test materializes nine artifacts as *real, freshly authored* git commits — seven forgeries plus two honest controls. Four forgeries touch the *wrong* files (an empty commit claiming the work; a keyword-stuffed subject; deleting the test that would have caught the gap; a long "production-ready" narration over a commit that did none of it). The other three are the adversary that matters — commits that touch the **exact deliverable files the plan names** but ship nothing behind them (empty stubs; code that does not import; a lookalike filename). Both channels see the *same* forgery-resistant evidence; the only asymmetry, stated up front so the fight is fair, is that the model judge *also* reads the agent's narration. The floor is a **ladder** of three rungs: the commit *subject* (forgeable, the author types it); *which files the commit touched* (a commit cannot fake that, but it certifies presence, not behavior — a stub passes); and the OS *running the test and reading the exit code* (a `0` the agent cannot forge, because it did not run the process).

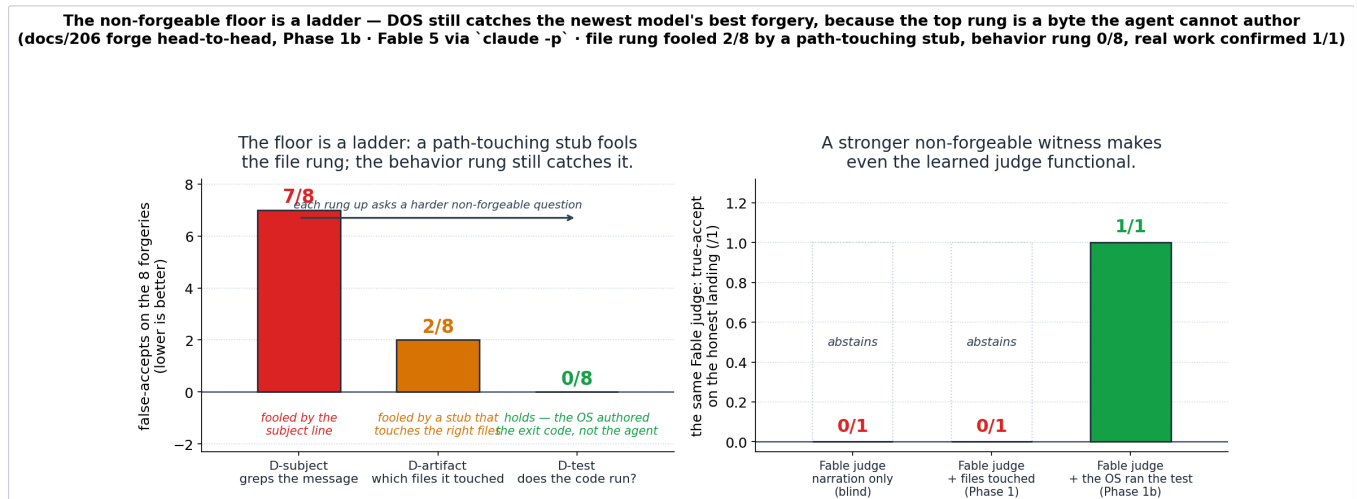


Figure A1. **The floor is a ladder, and its top rung still catches a frontier model's best forgery.** Left: on the 8 not-shipped artifacts, the grep-the-subject rung is fooled 7/8 and the presence rung 2/8 (path-touching stubs), but the behavior rung — the OS runs the test and reads the exit code — holds at 0/8. Right: on the one honest landing the same Fable judge abstains given narration or files-touched (0/1) but confirms it given the OS test result (1/1).

The result is not "the floor is perfect." The *presence* rung — what most people picture by "check that the files landed" — is **fooled 2 of 8** by the path-touching stubs, the honest ceiling of any file-counting check. What sits *above* it is the point: the **behavior rung holds at 0 of 8** and still confirms the one real landing, because it has the OS *run the code* and reads an exit code the agent cannot author (the forgeable subject rung is fooled on 7 of 8). The lesson is not "determinism wins" but **which non-forgable byte you condition belief on, and that the rung you must stand on rises with the forger's sophistication.**

The judge-side result is the more surprising one: the learned judge is *exactly as good as the evidence the kernel computes for it*. Given only narration, Fable abstains on the honest landing (0/1); given only which files were touched it *still* abstains (0/1) — it will not certify presence as completion. But given the OS test result — the byte the behavior rung reads — the *same model* confirms the work (1/1) and disagrees with every forgery (0/8). Left to its own reading the judge yields no value — gamed if forced to answer (35.2% false-accepts elsewhere), inert

if allowed to abstain. **The floor's value is that it computes the non-forgable witness — including running the code — for free, every time, lifting both the deterministic verdict and the learned one.** The stronger the model, the more the value concentrates in the byte it cannot author. (Full grid: docs/272.)

A.3. D.3 The payoff compounds: one block at the root keeps the whole downstream fleet clean

ΔB priced a *single* phantom propagating *one* hop. A real fleet is neither single nor shallow — a phantom is inherited again by the next agent, and a fleet *branches* — so the true quantity the gate protects is the whole set of downstream agents a root corruption reaches. We measured that set live, down a chain and across a fan-out tree, on the same tau2 airline witness with two arms (*believe*: each agent inherits the corrupt state; *adjudicate*: the gate blocked the root, so each inherits the gold state). The root corruption is one reservation R wrongly cancelled (injected; every *downstream* node runs live), and the witness is exact: **is R still cancelled when this agent finishes?**

F1: corruption compounds down a believe chain, the referee stops it at the root (tau2-bench · gemini-2.5-flash, live)

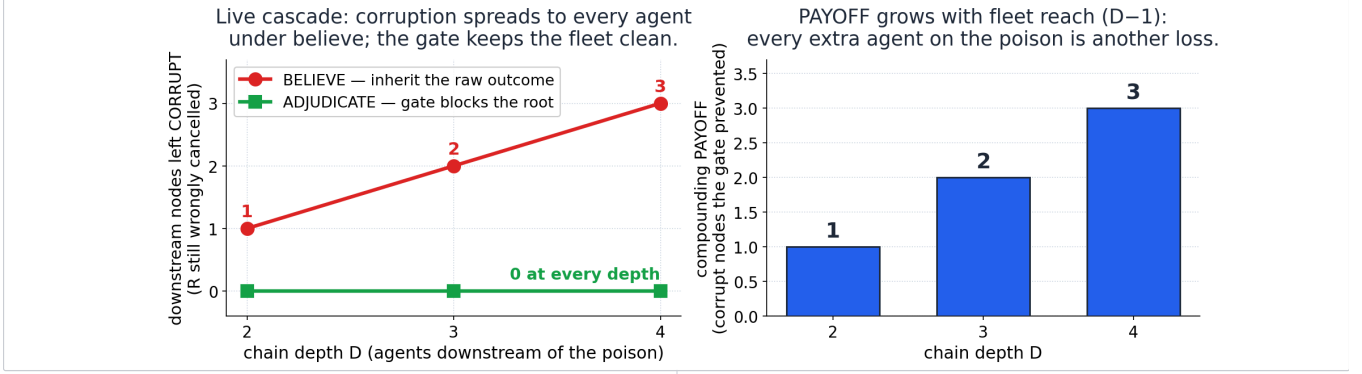


Figure A2. The cascade, run live, down a chain. Under **believe** all D-1 downstream agents inherit the cancelled reservation and are left corrupt (rising 1-2-3 with depth); under **adjudicate** the one root block keeps every node on the gold database (flat at zero). The believe agents *did not heal it* — a corrupt state is not second-guessed away the way a narrated claim is, which is why ΔB was near zero to a self-recovering peer.

Down a single chain the result is linear: under *believe* a depth-D chain leaves all D-1 downstream nodes corrupt, under *adjudicate* zero (the gate prevents 1, 2, and 3 corrupt nodes at depths 2, 3, 4). The believe agents are instructive: each, handed a cancelled reservation, simply could not do its task and gave up — none

restored R. So a capable peer re-checks and heals a *narrated* phantom (hence $\Delta B \approx 0$) but cannot re-check away a corrupt *starting world*; the state handoff is the one that compounds.

F1-super-linear: one over-claim caught at the root saves the whole fan-out tree (tau2-bench · gemini-2.5-flash, live)

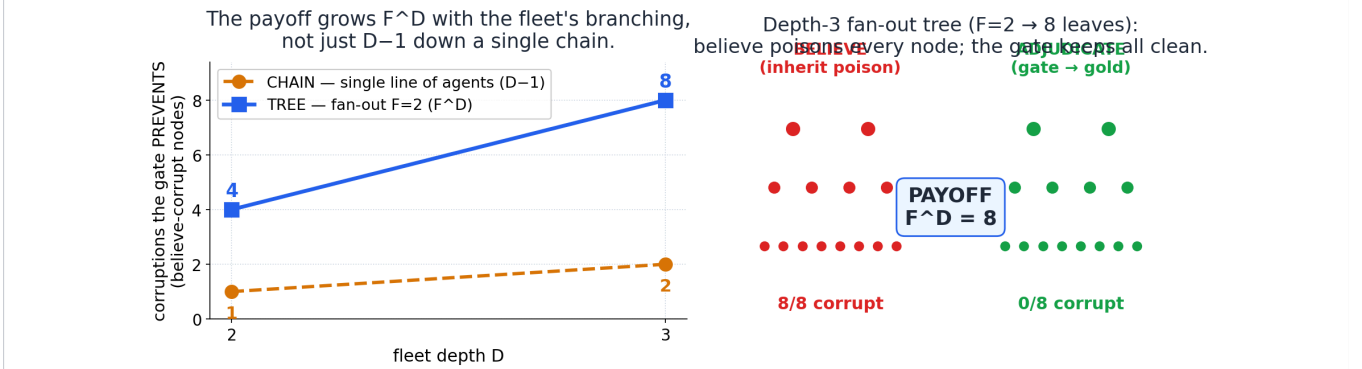


Figure A3. The same cascade across a fan-out tree ($F=2$). **Left:** the chain's linear D-1 payoff (1, 2) against the tree's F^D (4, 8). Branching makes the payoff super-linear in depth. **Right:** the depth-3 tree — under *believe* all 8 leaves are corrupt; under *adjudicate* the one block at the root keeps all 8 clean.

A chain understates a fleet, because fleets branch. A depth-D, fan-out-F dependency *tree* has F^D leaves, all downstream of the root corruption. Under *believe* every leaf is corrupt (**4 of 4** at depth 2, **8 of 8** at depth 3, $F=2$); under *adjudicate* the single root block keeps every leaf clean. So the prevented-corruption payoff is F^D — 4 at depth 2, 8 at depth 3 — against the chain's linear D-1. F^D here is **breadth** (F^D agents each *blocked* by the one shared poison, not one corrupt value mutating into many): a poisoned shared resource fails every agent that depends on it, and that set grows F^D as the fleet branches. This is the *depth* term ΔB pinned to one, now measured live. (Caveats: the root is injected, not waited for, and each node re-inherits the same root state rather than chaining onto the previous node's output; it is an existence result for the compounding *shape* — one model, $F=2$, depths 2-4 — not a calibrated constant. The fleet-of-one floor holds: at depth 0 the payoff is zero.)

11. References

- [1] Barres, V. and Dong, H. and Ray, S. and Si, X. and Narasimhan, K.. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982, 2025.
- [2] {METR}. Measuring AI Ability to Complete Long Software Tasks. arXiv:2503.14499, 2025.
- [3] Pathak, D. and others. Detecting Silent Failures in Multi-Agent AI Trajectories. arXiv:2511.04032, 2025.
- [4] Meir, T. (Meirtz). Limen: Coordination for Concurrent Autonomous AI Agents over Shared Mutable State (advisory write-leases over MCP). Software artifact, github.com/Meirtz/Limen, 2025.
- [5] Pugachev, S.. CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation. arXiv:2510.18893, 2025.
- [6] Anderson, J. P.. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972.
- [7] Wald, A.. Sequential Tests of Statistical Hypotheses. Annals of Mathematical Statistics 16(2):117-186, 1945.
- [8] Page, E. S.. Continuous Inspection Schemes. Biometrika 41(1/2):100-115, 1954.
- [9] Mohan, C. and Haderle, D. and Lindsay, B. and Pirahesh, H. and Schwarz, P.. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. ACM Transactions on Database Systems 17(1):94-162, 1992.
- [10] Bernstein, P. A. and Hadzilacos, V. and Goodman, N.. Concurrency Control and Recovery in Database Systems. Addison-Wesley, 1987.
- [11] Bishop, M. and Dilger, M.. Checking for Race Conditions in File Accesses. Computing Systems 9(2):131-152, 1996.
- [12] Lambert, N. and others (Allen Institute for AI). Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124, 2024.
- [13] Bowman, S. R. and others (Anthropic). Measuring Progress on Scalable Oversight for Large Language Models. arXiv:2211.03540, 2022.
- [14] Baker, B. and Huizinga, J. and Gao, L. and others (OpenAI). Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025.